

Universidad Autónoma de Madrid

Escuela Politécnica Superior



Grado en Ingeniería Informática

TRABAJO DE FIN DE GRADO

SIMULACIÓN DE MODELOS EVOLUTIVOS EN CÁNCER: TERAPIA ADAPTATIVA Y OTRAS EXTENSIONES

Javier López Cano
Tutor: Ramón Díaz Uriarte

Junio 2021

SIMULACIÓN DE MODELOS EVOLUTIVOS EN CÁNCER: TERAPIA ADAPTATIVA Y OTRAS EXTENSIONES

Autor: Javier López Cano
Tutor: Ramón Díaz Uriarte

Dpto. de Bioquímica
Escuela Politécnica Superior
Universidad Autónoma de Madrid

Junio 2021

Agradecimientos

A mi familia y amigos por su apoyo durante estos meses.

A Ramón Díaz Uriarte por su dedicación y guía a lo largo de este proyecto.

“Lo que importa verdaderamente en la vida no son los objetivos que nos marcamos, sino los caminos que seguimos para lograrlo.”

Peter Bamm

Abstract

Abstract — The increasing frequency of cancer cases, as well as their severity, has led the scientific community to amplify its efforts on researching this illness. Due to this increased interest, numerous tools have appeared to aid with this purpose, some of them focused on predicting the evolution of cancerous tumours, known as tumoral progression simulators.

On this project, the functionality of one of these simulators, **OncoSimulR** is going to be extended. A group of changes is going to be added to the program in order to allow the user to specify arbitrarily complex user variables that depend on existing parameters of the simulation, giving the user the possibility of tracking any data of the tumoral progression. These variables will also be usable for the definition of the interventions that the program can emulate, adapting them to the current state of the tumor in a given moment; this will allow us to simulate adaptive therapy, a promising new approach to therapy.

Furthermore, some tests will be developed in order to ensure the optimal performance of this new functionality. Lastly, examples of some use cases of the program will be given, illustrating the new possible ways to use the tool.

Key words — Cancer, tumor, tumoral progression simulator, OncoSimulR, user variables, interventions, adaptive therapy, evolution.

Resumen

Resumen — La creciente preocupación por la proliferación del cáncer como enfermedad, así como por la gravedad de este, ha propiciado que en los últimos años se hayan multiplicado los esfuerzos en su investigación. Con este fin han surgido numerosas herramientas para predecir la evolución de un cáncer, conocidas como simuladores de progresión tumoral.

En este trabajo se va a ampliar la funcionalidad de uno de estos simuladores, **OncoSimulR**. Sobre éste se va a introducir la posibilidad de que el usuario defina una serie de variables de usuario tan complejas como desee, en función de parámetros de la simulación, que permitirán al usuario obtener prácticamente cualquier dato que quiera acerca de la evolución del tumor. Además se va a permitir que estas variables se empleen en la definición de las intervenciones que esta aplicación permite reproducir, con el fin de adaptar estas al estado del tumor en cada momento, logrando de este modo emular la conocida como terapia adaptativa, un prometedor nuevo tipo de terapia.

Tras explicar en qué consisten los cambios llevados a cabo para esto, se describirá la batería de pruebas empleada para comprobar su correcto funcionamiento, y se mostrarán algunos ejemplos prácticos del uso del programa, ilustrando esta nueva funcionalidad.

Palabras clave — Cáncer, tumor, simulador de progresión tumoral, OncoSimulR, variables de usuario, intervenciones, terapia adaptativa, evolución.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Estructura del documento	3
2. Estado del Arte. OncoSimulR	5
2.1. Modelos de simulación estocástica. Algoritmo BNB de Mather et Al	8
2.2. Ramas con versiones divergentes	9
2.2.1. Birth y death rate	10
2.2.2. Intervenciones	11
3. Diseño	13
3.1. Introducción. Terapia adaptativa y variables de usuario	13
3.2. Definición de variables y reglas	14
3.3. Algoritmo de actualización	15
3.4. Definición de la estructura de salida	16
4. Desarrollo	17
4.1. Introducción	17
4.2. Unificación de Ramas	18
4.3. Optimización del código de intervenciones	19
4.4. Implementación de variables de usuario y reglas	20
4.4.1. En R	20
4.4.2. En C++	22
4.5. Integración en el flujo principal de código	26
4.6. Variables de usuario en las intervenciones. Terapia adaptativa	28
5. Pruebas y Resultados	29
5.1. Comprobación de funcionalidad previa	29
5.2. Tests de nueva funcionalidad	30
5.2.1. Variables de usuario	31
5.2.2. Intervenciones con variables de usuario	31
5.3. Resultados	32
5.3.1. Variables de usuario como datos de interés a la salida	32
5.3.2. Terapia Adaptativa	34

6. Conclusiones y trabajo futuro	39
6.1. Conclusiones	39
6.2. Líneas de trabajo futuro	39
Bibliografía	41
Glosario	43
Apéndices	45
A. Código para reproducir los ejemplos de la sección Resultados	47
A.1. Ejemplo de uso de variables de usuario para controlar la proporción de cada genotipo en la población total	47
A.2. Ejemplo de uso de variables de usuario para calcular la diferencia entre tasa de nacimiento y muerte de los genotipos	49
A.3. Ejemplo de terapia adaptativa para atacar al genotipo más adaptado . . .	51
A.4. Ejemplo de terapia adaptativa en escenario verosímil	53
B. Documentación de las principales funciones de R	57
C. Cabeceras del TAD de variables de usuario	59

Índice de figuras

2.1. Ilustración del funcionamiento terapia adaptativa	7
5.1. Evolución poblacional. Primer ejemplo variables de usuario	33
5.2. Proporción de genotipos en la población. Primer ejemplo variables de usuario	33
5.3. Diferencia tasas de nacimiento y muerte. Segundo ejemplo variables de usuario	34
5.4. Evolución poblacional. Segundo ejemplo variables de usuario	34
5.5. Diferencia tasas de nacimiento y muerte. Primer ejemplo terapia adaptativa	36
5.6. Evolución poblacional. Primer ejemplo terapia adaptativa	36
5.7. Evolución poblacional. Segundo ejemplo terapia adaptativa	38

Índice de tablas

2.1. Modelos evolutivos OncoSimulR	6
5.1. Variables del segundo ejemplo de terapia adaptativa.	37
5.2. Reglas del segundo ejemplo de terapia adaptativa.	37

1

Introducción

1.1. Motivación

El cáncer[1] es una de las enfermedades más comunes y preocupantes en la actualidad. En el año 2020, supuso en España la segunda causa de muerte más común, después de las enfermedades circulatorias y por encima de las infecciosas (que incluyen los casos de la pandemia de Covid-19)[2]. Estos datos explican que el interés por la enfermedad así como los esfuerzos en su investigación hayan ido en aumento durante los últimos años.

A través de la investigación se han logrado encontrar diversos tratamientos, intervenciones quirúrgicas y terapias que permiten mejorar el pronóstico de distintos tipos de cáncer, e incluso eliminarlos en algunos casos. Estos tratamientos, sin embargo, no siempre son viables o totalmente efectivos, por lo que en ocasiones es necesario plantear un enfoque distinto, que, en lugar de centrarse en eliminar el mayor número posible de células cancerígenas, se centre en tratar de frenar la evolución del tumor, de modo que su crecimiento sea lo más lento posible. Es aquí donde aparece, entre otras posibilidades, la llamada **terapia adaptativa**[3][4].

La terapia adaptativa es una estrategia de aplicación de tratamientos contra el cáncer (y también en algunas enfermedades causadas por bacterias), centrada en aplicar tratamientos distintos dependiendo de la etapa en la que se encuentre el desarrollo del tumor, con el fin de reducir su avance al mínimo posible. Esta terapia, además, trata de explotar la competencia que se da entre distintos tipos de células tumorales [4][5]. De este modo, se va monitorizando el estado del tumor para llevar a cabo la opción más prometedora en cada momento. Al aplicar una diversidad de tratamientos, y hacer uso de la competencia entre tipos celulares, se logran evitar algunos de los problemas más comunes de los tratamientos más clásicos de cáncer, como es por ejemplo la aparición de

células tumorales resistentes a algunos de estos. Un ejemplo de ello suele ocurrir con la quimioterapia, ya que, en muchas ocasiones, ésta no logra eliminar la totalidad del tumor pues algunas células pueden desarrollar una mutación que las haga menos susceptibles a ésta. Es en este momento cuando entra en juego la estrategia de la terapia adaptativa, variando el tratamiento y aplicando otro como puede ser la radioterapia, logrando frenar el avance de estas resistentes, al alterar también las relaciones de competencia entre células y evitar seleccionar células resistentes al primer antitumoral.

Es notoria, por tanto, la importancia de disponer de mecanismos para “predecir” la evolución de un tumor, con el fin de poder planificar qué medidas llevar a cabo para eliminarlo o, si no es posible, frenar su avance. En este punto es dónde entran en juego los simuladores de progresión tumoral; estos son herramientas informáticas que permiten simular la evolución de un grupo de células tumorales dadas unas condiciones iniciales concretas.

Entre estos simuladores, destaca la herramienta **OncoSimulR**[6], sobre la que se va a enfocar este trabajo. Esta herramienta implementa, además de la funcionalidad base para simular la progresión tumoral, funcionalidad extra, como la capacidad de emular los efectos de algunos tratamientos, o la posibilidad de especificar el fitness de los distintos genotipos celulares siguiendo una gran variedad de estrategias, permitiendo simulaciones tremendamente complejas y específicas. Sin embargo, esta herramienta no permite aún la simulación de terapia adaptativa, es en esta funcionalidad en la que se centrará principalmente este trabajo.

1.2. Objetivos

Como hemos comentado en el apartado anterior 1.1, el principal objetivo de este trabajo es lograr que el programa **OncoSimulR** pueda simular de forma efectiva los efectos de la terapia adaptativa en la evolución de un tumor. Es decir, se debe permitir que las intervenciones que ya se pueden definir en este programa, puedan variar en función del estado en el que se encuentra el tumor a lo largo de la simulación. Para ello, se deben llevar a cabo distintas acciones que vamos a desglosar en subobjetivos específicos:

- Familiarizarse con la herramienta OncoSimulR en el estado en que se encuentra previamente al inicio del trabajo (especialmente el flujo general de código de la simulación y el funcionamiento de las intervenciones, que son los aspectos más relacionados con la funcionalidad que se quiere implementar).
- Aprender sobre R y C++, ya que son dos lenguajes con los que no estoy totalmente familiarizado al inicio de este trabajo.
- Completar la unificación de las ramas anteriores solucionando todos los errores que aparezcan durante el proceso.
- Añadir la posibilidad de definir variables de usuario arbitrariamente complejas (que sean función de los parámetros de la simulación tales como población total,

subpoblaciones de los genotipos, fitness, etc) tanto como salida en sí mismas, como para emplearse en las intervenciones. Este será el mecanismo principal que permitirá la implementación de la terapia adaptativa, como veremos en el apartado 3.

- Insertar la funcionalidad de las variables de usuario en el flujo general de código de la simulación.
- Permitir que las intervenciones empleen estas nuevas variables de usuario. Esto es lo que haría posible simular la terapia adaptativa.
- Generar una batería de pruebas extensa para comprobar el correcto funcionamiento de las variables de usuario y las intervenciones que las emplean.

1.3. Estructura del documento

A lo largo de este documento se explicará en detalle todo el trabajo llevado a cabo para lograr los objetivos definidos en el punto anterior. Debido a la extensión de este, se dividirá la memoria en diversos apartados:

- En primer lugar, en 2 analizaremos el estado del arte, centrándonos en el punto en que se encuentra previamente a este trabajo la herramienta **OncoSimulR**[6], su funcionalidad disponible y el funcionamiento del algoritmo de simulación que implementa, así como el análisis detallado de las dos ramas con funcionalidad recientemente implementada que deberán unificarse.
- A continuación, en el apartado de diseño 3, veremos la estrategia definida para alcanzar la funcionalidad deseada en el programa, y como se ha planificado cada paso que se ha de llevar a cabo para lograrlo.
- Posteriormente, en el apartado de implementación 4, abordaremos el proceso de modificación del código del programa llevado a cabo para implementar los cambios definidos en el apartado de diseño. Este apartado incluye en detalle el proceso de unificación de ramas divergentes 4.2 que se debió realizar previamente a la implementación de la nueva funcionalidad, que fue un proceso bastante más complejo de lo esperado inicialmente.
- Seguidamente, en el apartado de pruebas y resultados 5, explicaremos los tests llevados a cabo para asegurar el correcto funcionamiento de lo implementado, e ilustraremos la nueva funcionalidad con una serie de ejemplos de ejecución con sus correspondientes resultados.
- Por último, en el apartado 6 extraeremos unas breves conclusiones acerca del proyecto, y analizaremos las posibles líneas de trabajo futuro para extender aún más esta funcionalidad.

Cabe destacar que, si bien una de las partes más complejas del trabajo ha sido la unificación de las ramas anteriores, debido a los problemas surgidos de esta, este tema no aparece reflejado en el apartado de diseño, puesto que no ha tenido una parte extensa de planificación, ya que no se tenía previsto que fuese a ser una tarea tan compleja como acabó siendo. Por tanto este tema se tratará directamente en el apartado de implementación, explicando de forma detallada los problemas detectados y cómo se han solucionado.

2

Estado del Arte. OncoSimulR

Hemos visto en el apartado 1 que la principal motivación de este proyecto es lograr reproducir la terapia adaptativa durante un proceso de simulación de progresión tumoral. Para ello vamos a trabajar sobre el paquete **OncoSimulR** [6] de R que implementa un simulador de este tipo.

Los simuladores de progresión tumoral son programas que permiten reproducir de forma artificial la evolución de un grupo de células tumorales dadas unas condiciones iniciales concretas. Son de gran interés en el ámbito de la biología pues permiten la obtención de una gran cantidad de datos acerca de la evolución de modelos tumorales excesivamente complejos para su análisis matemático. Permiten también en algunos casos (como es el caso de **OncoSimulR**) emular los efectos que tendrían ciertos tratamientos o variaciones en las condiciones externas en la progresión de dichos tumores.

Dentro de los simuladores disponibles encontramos **OncoSimulR**, un paquete de R incluido dentro del proyecto BioConductor[7]. BioConductor es un proyecto que fomenta el desarrollo de software de código abierto para el análisis de estudios biológicos, fundamentalmente en el área de la bioinformática. **OncoSimulR** ha sido desarrollado por Ramón Díaz Uriarte, Profesor Titular del Dpto. de Bioquímica de la Universidad Autónoma de Madrid, con el fin de simular la evolución de conjuntos de células tumorales con reproducción asexual, aunque actualmente esta funcionalidad se ha extendido permitiendo simulaciones mucho más complejas, emulando modelos casi arbitrarios. Además, una de las ramas más recientes, permite reproducir los efectos de ciertos tratamientos para evaluar su efectividad en la ralentización del avance del tumor.

Este paquete está escrito en los lenguajes R y C++. Con R se logra una interfaz amigable en un lenguaje con el que están más familiarizados los usuarios de aplicaciones bioinformática. Implementando en C++ los algoritmos de la simulación se consigue una

buena eficiencia en la ejecución de código, porque son estos algoritmos los que tienen una mayor complejidad y coste computacional. Actualmente la funcionalidad de **OncoSimulR** incluye:

- Especificar la adaptabilidad de los genes mediante el “fitness” (“eficacia biológica”) y las tasas de nacimiento y tasas de muerte con la función **allFitnessEffects**, en esta función se permite:
 - Especificar el fitness de cada genotipo siguiendo los siguientes modelos¹:

Modelo	Tasa de Nacimiento	Tasa de Muerte
Exponencial	Definido por usuario	1
McFarland[9]	Definido por el usuario	Proporcional a la población (hay competencia)

Tabla 2.1: Modelos evolutivos OncoSimulR

- De igual modo, se puede decidir no emplear estos modelos y asignar de forma explícita tanto la tasa de nacimiento como la tasa de muerte (En una de las ramas con funcionalidad más avanzada que la principal, explicada en 2.2.1)[10]
- Para especificar estos valores de fitness se pueden emplear distintas estrategias:
 - Definir un fitness variable para los genotipos en función de la frecuencia de estos, es decir, según la cantidad de población (frecuencia absoluta), o la proporción frecuencia relativa) de cada genotipo que haya presente en el conjunto de células simulado.
 - Determinar la existencia o no de epistasia entre los genes, es decir, si la presencia de un determinado gen mutado favorece a otro, de este modo, el fitness de una célula con un determinado gen mutado aumentará si muta otro gen con el que tiene epistasia.
 - Especificar los valores del fitness o de las tasas de nacimiento y muerte de forma explícita introduciendo directamente sus valores. En una rama explicada en el punto 2.2.1.
 - Especificar variaciones en el fitness en función del orden de mutación de los genes, es decir, permitir que un genotipo con dos genes A y B mutados tenga fitness distintos si el gen A ha mutado primero, o lo ha hecho el genes B. Esto se realiza mediante un DAG (grafo dirigido acíclico).
- Emplear este fitness para la simulación de la progresión tumoral, definiendo para esta además del fitness, las tasas de mutación iniciales, la población inicial y la presencia o no de genes ya mutados desde el inicio de la simulación. Esto se realiza mediante la función **OncoSimulIndiv**. Para realizar esta simulación el programa emplea el modelo **BNB de Mather et al**[11] que se explicará en el punto 2.1.

¹Se permite también el modelo de Bozic[8] un antiguo modelo, que ya no se emplea, en el que la tasa de nacimiento es 1 y la de muerte se especifica por el usuario en función del genotipo

- Emular los efectos de ciertos tratamientos o modificaciones en las condiciones externas en la evolución del tumor. Esto se hace mediante la funcionalidad de “intervenciones” implementada en otra de las ramas con funcionalidad extra sobre la rama principal; esta se explicará con más detalle en el punto 2.2.2.
- Mostrar mediante gráficos la evolución de prácticamente todos los datos de la simulación.
- Ejecución de varias simulaciones al mismo tiempo, permitiendo además la paralelización de estas simulaciones. Para ello se emplean las funciones **OncoSimulPop** y **OncoSimulSample**.

Toda esta funcionalidad permite una gran adaptabilidad al programa, haciéndolo muy útil para la simulación de escenarios muy complejos y sofisticados al permitir la especificación de forma prácticamente arbitraria de todas las condiciones de partida de la simulación. Pero aún falta la posibilidad de intervenir en las simulaciones arbitrariamente durante la ejecución para simular escenarios como la terapia adaptativa. Es en esto en lo que se centrará este trabajo.

Este tema se ilustra de forma muy detallada en los artículos de Hansen[4][5]. En estos papers se plantean numerosos escenarios que hasta ahora no eran simulables en OncoSimulR:

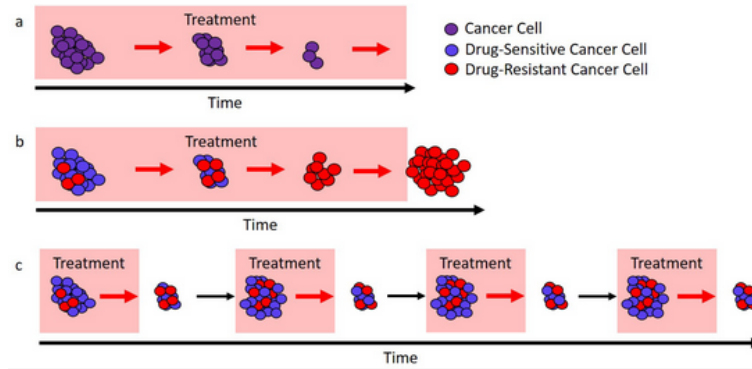


Figura 2.1: Ilustración del funcionamiento terapia adaptativa

En esta ilustración vemos los casos **a** y **b**, sin terapia adaptativa, que serían simulables actualmente en **OncoSimulR**. El primero consistiría en simular un tratamiento continuo que, en esta situación ideal, sería efectivo contra todas las células del tumor. El segundo ilustra el mismo caso, pero esta vez hay células en el tumor resistentes a este tratamiento (esto se simularía en **OncoSimulR** mediante una intervención que solo afectase a un genotipo), en este caso vemos que las células resistentes aumentan notablemente el tamaño del tumor a pesar del tratamiento. Por último encontramos el caso **c** que ilustra un caso de terapia adaptativa, donde se monitoriza el avance del tumor y se aplica el tratamiento en momento seleccionados para lograr mantener constante el tamaño del tumor y balanceada la población de ambos genotipos. Es de gran importancia mantener estas poblaciones balanceadas para garantizar la competencia celular que, como nos explica Hansen[4][5]

en sus artículos, garantiza que la población resistente al tratamiento no se dispare, al tener que competir por los recursos con las células no resistentes, de este modo se logra mantener la población total controlada. Este último escenario no era simulable hasta ahora en **OncoSimulR**, al no poderse modificar las intervenciones en función del desarrollo de la simulación, para asegurar no eliminar completamente el genotipo sensible y garantizar la competitividad celular.

Existen además casos más complejos de terapia adaptativa, dónde se varía el tratamiento en función del estado del tumor, aplicando algunos más o menos agresivos según sea necesario. El objetivo del trabajo es por tanto lograr la simulación en **OncoSimulR** de los escenarios de la terapia adaptativa.

2.1. Modelos de simulación estocástica. Algoritmo BNB de Mather et Al

La simulación de progresión en modelos tumorales requiere emplear un modelo estocástico en tiempo continuo, que sea lo más exacto posible, pero, a la vez, relativamente rápido, pues debe ser ejecutado en un tiempo razonable. Para ello, OncoSimulR[6] emplea, como hemos mencionado en el apartado 2 el algoritmo **BNB de Mather et al**[11], basado en algoritmos anteriores como los **Direct Method** y **First Reaction Method** de Gillespie[12] y el **Next Reaction Method** de Gibson y Bruck[13], pero aplicando diversas mejoras sobre ellos.

De estos algoritmos el primero en aparecer fue el **Direct Method** de Gillespie[12], que se basa en, dada una determinada situación, hallar cual es la próxima interacción que va a ocurrir y cuándo se va a producir. Para ello genera dos números aleatorios en cada iteración mediante distribuciones exponenciales, uno que determina cual será la próxima interacción, y otro que determina cuándo se producirá esta.

Posteriormente el propio Gillespie[12] realizó ciertas modificaciones a este algoritmo dando lugar al **First Reaction Method**, en este caso, en cada iteración no se determina cual será la próxima interacción, sino que directamente se determinan los tiempos en que se producirá la siguiente interacción de cada tipo suponiendo que no se produce otra antes. De este modo, obtendríamos un tiempo para cada posible interacción. Tras esto se selecciona el tiempo menor, y este determina la interacción que se llevará a cabo. Para este proceso se deben generar tantos números aleatorios como posibles interacciones haya, puesto que cada uno dará lugar a uno de los tiempos mediante una distribución exponencial.

Gibson y Bruck mejoran aún más este proceso con su "Next Reaction Method"[13]. En este parten del "First Reaction Method."e incluyen en este unas colas de prioridad y grafos de dependencias para mejorar su rendimiento. Los grafos de dependencias son unos grafos dirigidos que determinan qué interacciones dependen de otras, de modo que cuando una interacción se produce podemos saber mediante el grafo, cuales se ven afectadas y cuales no. Con esto se consigue reaprovechar los tiempos calculados previamente para

algunas de las interacciones cuando estas no se ven afectadas por la última que se ha producido. Junto a esto, introducen colas de prioridad para las interacciones, donde la prioridad es el tiempo que tienen asignado, de este modo la siguiente interacción que se producirá es la primera en la cola. En general con esto logran que en cada iteración solo haya que recalcular los tiempos correspondientes a las interacciones que se ven afectadas por la última que se ha producido, reduciendo significativamente el número de números aleatorios a generar en cada iteración, y por tanto, mejorando el rendimiento.

Por último, Mather et Al aplican estos algoritmos de forma más directa a la evolución de poblaciones sometidas a las interacciones de mutación, muerte y nacimiento en su algoritmo **BNB (Binomial Negative Binomial)**[11]. Este se basa en el uso de distribuciones binomiales y binomiales negativas para la determinación de las poblaciones. Funciona en rasgos generales de forma similar al de Gibson y Brucks, pero tiene la particularidad de que únicamente presta atención a los tiempos de las mutaciones, por ser la interacción que induce cambios más significativos en este tipo de poblaciones. De este modo las poblaciones tan solo se actualizan cuando se produce una mutación, ignorando los eventos de nacimiento y muerte hasta ese momento.

Este algoritmo es el seleccionado para OncoSimulR por ser altamente eficiente para las simulaciones en cáncer, donde los eventos de mutación son notablemente menos frecuentes que los nacimientos y las muertes. Sin embargo este únicamente da resultados exactos cuando las tasas de nacimiento muerte y mutación son constantes durante toda la simulación, si esto no se cumple, tan solo se puede aplicar una versión aproximada de este algoritmo.

2.2. Ramas con versiones divergentes

Además de la funcionalidad básica de simulación que proporciona el algoritmo **BNB de Mather et al** explicado, **OncoSimulR** permite simulaciones más complejas mediante la adición de otra funcionalidad a este algoritmo. Algunos ejemplos de esto son la gran libertad para la definición de las tasas de nacimiento y tasas de muerte, o la posibilidad de simular el resultado de ciertos tratamientos, como se ha mencionado brevemente con anterioridad.

Estas funcionalidades, al ser muy recientes y haberse desarrollado simultáneamente, se encuentran, al inicio de este trabajo, en dos ramas diferentes del programa, cada una implementando una de las dos posibilidades, que deben ser unificadas para proseguir con la nueva funcionalidad, pues se basa en parte en lo implementado en estas. Procederemos ahora a explicar brevemente las peculiaridades y funcionalidad concreta de estas dos ramas.

2.2.1. Birth y death rate

En esta rama[10] se implementa la funcionalidad que permite definir de forma manual y arbitraria las tasas de nacimiento y muerte. La funcionalidad anterior a esta rama tan solo permitía la definición de un único parámetro “fitness”, que representaba la tasa de muerte (en el modelo de **Bozic**) o la tasa de nacimiento (en los modelos **Exponencial** y de **McFarland**) según lo indicado en la tabla 2.1, pero no se podían definir ambos de forma manual al mismo tiempo. Para incluir esto, esta rama contiene numerosos cambios en funciones básicas del algoritmo de simulación, que incluyen:

- **allFitnessEffects**: esta función es la que permite al usuario definir el fitness de los genotipos, por tanto, en esta rama, contiene cambios que permiten definir no solo un parámetro para cada genotipo (que antes era el fitness), sino dos, correspondientes a las tasas de nacimiento y muerte respectivamente.
- **evalAllGenotypes**: esta función se emplea para obtener los valores del fitness de todos los genotipos, por tanto en esta rama, implementa cambios para devolver dos valores para cada genotipo (correspondientes a las tasas de nacimiento y muerte) en lugar de uno solo, como previamente. Para ello incluye cambios además en otras funciones internas que gestionan el almacenamiento de los datos en las estructuras de C++.
- **OncoSimulIndiv**: la función principal de simulación se ve también afectada, pues debe permitir la simulación mediante un modelo distinto a los de **Bozic**, **Exponencial** y **McFarland** que ya se permitían, pues estos no necesitan definir ambas tasas. Por tanto se modifica esta función para introducir un nuevo modelo de simulación “Arb” (arbitrario) que permite introducir ambos valores.

Además, la rama incluye también modificaciones en numerosas estructuras en C++, especialmente en

- **fitnessLandscape_struct**: estructura que mapea los genotipos a su fitness o, en esta rama, tanto a su tasa de nacimiento como de muerte.
- **fitnessEffectsAll**: estructura que contiene todos los datos para manejo del fitness, ahora debe incluir datos tanto para la tasa de nacimiento como de muerte.
- **spParamsP**: clase que almacena los datos de cada genotipo, tasa de mutación, fitness, población, etc. En esta rama en lugar del fitness se incuyen por separado datos sobre las tasas de nacimiento y de muerte.

Al modificarse estas estructuras, se introducen también ligeros cambios en todas las funciones que las emplean, que deben también tenerse en cuenta.

2.2.2. Intervenciones

La funcionalidad correspondiente a la simulación de tratamientos mediante lo que en la aplicación **OncoSimulR** se llaman “intervenciones” está implementada en otra rama separada[14]. En ella se permite definir ciertos cambios en la población o el fitness de uno o varios genotipos, que se llevarán a cabo cuando se cumpla una determinada condición o “trigger”. De este modo se puede emular el efecto que tendrían algunos tratamientos o intervenciones quirúrgicas en el desarrollo del tumor.

Las “intervenciones” están definidas por los siguientes parámetros:

- **ID**: identificador de una intervención que la diferencia de otras intervenciones.
- **Trigger**: expresión booleana que determina la condición que debe cumplirse en la simulación para que la intervención se lleve a cabo.
- **WhatHappens**: expresión que determina el cambio que se produce al ejecutarse la intervención. Puede ser un cambio en la población total, en el tamaño de población de uno o varios genotipos, o una variación en el fitness de alguno de ellos.
- **Periodicidad**: tiempo mínimo que debe pasar antes de que la misma intervención pueda ejecutarse de nuevo en caso de que esta se deba llevar a cabo varias veces.
- **Repeticiones**: número máximo de veces que puede ejecutarse la intervención a lo largo de la simulación.

Partiendo de esta definición, en la rama se implementa un algoritmo en la función **executeInterventions** del fichero **interventions.cpp** que controla la ejecución de las intervenciones mediante el proceso:

- Para cada intervención:
 - Si se cumple el **Trigger**:
 - Si ha pasado el tiempo mínimo de **Periodicidad** y quedan **Repeticiones** para la intervención:
 - ◊ Se ejecutan los cambios definidos en **WhatHappens**.

Este algoritmo luego se inserta en el flujo principal de código de la simulación, para que en cada ciclo de esta se compruebe si se deben ejecutar y en caso afirmativo se lleven a cabo.

En esta rama también se incluyen una nueva función de R que permite la definición de las propias intervenciones por parte del usuario, esta es:

```
createInterventions <- function(interventions , genotFitness ,  
  ↪ frequencyType = "auto")
```

Que realiza las comprobaciones necesarias para asegurar que la definición de las intervenciones por parte del usuario es correcta, y prepara los datos de estas para pasarlos a C++.

El usuario definirá las intervenciones en R mediante una lista con todas las intervenciones que desee especificar, cada una de las cuales será a su vez otra lista, con los atributos necesarios para definirla. Un ejemplo de esta definición es:

```
interventions <- list(  
  list(ID           = "intOverA",  
        Trigger      = "n_B >= 5",  
        WhatHappens   = "n_A = n_A * 0.1",  
        Repetitions   = 0,  
        Periodicity   = Inf  
  ), list(  
    ID           = "intOverA2",  
    Trigger      = "(n_B >= 5) and (T > 4)",  
    WhatHappens   = "n_A = n_A * 0.4",  
    Repetitions   = Inf,  
    Periodicity   = Inf  
  )  
)
```

En este ejemplo hemos definido dos intervenciones. En la primera cuando la población del genotipo B llega a 5 (como indica el “Trigger”), se reduce la población del genotipo A al 10 % de su población anterior (como muestra el “WhatHappens”). En la segunda, cuando la población de A llega a 5, y han pasado más de 4 segundos de simulación, se reduce la población de A al 40 % de su población anterior.

De igual modo, se incluye también en la rama la definición de varias estructuras en C++ para almacenar los datos correspondientes a estas intervenciones, estas estructuras son **Interventions** que contiene los datos de cada intervención, e **InterventionsInfo** que contiene los datos para gestionar el conjunto de intervenciones definidas para una simulación. Ambas estructuras están definidas en **interventions.h**

Por último, esta rama contiene también modificaciones ligeras en la función principal de simulación **OncoSimulIndiv** y muchas de sus sub-funciones tanto en R como en C++ para que estas tomen el argumento **interventions** de forma adecuada.

3

Diseño

3.1. Introducción. Terapia adaptativa y variables de usuario

Como hemos visto en la sección 1.1, la base fundamental de la terapia adaptativa es la posibilidad de crear tratamientos específicos para determinados momentos en el desarrollo de un tumor. Dentro del programa **OncoSimulR** los tratamientos están simulados a través de las **intervenciones**. Por tanto, para simular la terapia adaptativa debemos lograr que estas intervenciones puedan depender del estado de desarrollo del tumor, es decir, del estado de la simulación en el momento concreto en que se va a ejecutar la intervención.

Para ello se ha decidido añadir al programa la posibilidad de definir variables de usuario, es decir, se ha incluido la posibilidad de que el usuario defina una serie de variables cuyo valor sea una función arbitrariamente compleja de distintos parámetros de la simulación. A lo largo de la simulación tumoral, estas variables se irán actualizando de acuerdo a la fórmula introducida por el usuario, haciendo que los valores vayan variando en función de la evolución del tumor.

Estas variables deberán poder ser empleadas para elaborar las expresiones que definen la acción a realizar en una intervención (explicadas en el punto 2.2.2), permitiendo así condicionar el efecto de una intervención al estado del tumor en el momento de la ejecución de esta, es decir, permitiendo la terapia adaptativa.

Con estos requerimientos presentes, procedemos al diseño de estas variables de usuario.

3.2. Definición de variables y reglas

A la hora de plantearse el diseño necesario para esta funcionalidad, lo primero que necesitamos saber es qué son y como se definen las variables de usuario que vamos a implementar.

Las variables de usuario son variables arbitrariamente complejas que un usuario puede definir a partir de combinaciones o funciones de algunas variables ya existentes en el programa. Para obtener estas variables arbitrariamente complejas vamos a necesitar de una serie de reglas que determinen el valor de estas variables en cada momento de acuerdo a las condiciones propuestas por el usuario para cada una de ellas.

A la hora de definir estas variables de usuario se ha buscado un diseño elegante y amigable con el usuario, dando a este libertad para establecer los nombres de las variables de forma arbitraria, de modo que puedan seleccionarlo de acuerdo a sus preferencias.

Por tanto podemos determinar que por un lado tendremos un listado de variables, cada una de las cuales deberá contener:

- **Nombre:** nombre de la variable de usuario que permite identificarla y diferenciarla de el resto de variables definidas.
- **Valor:** valor numérico de la variable en cada momento.

Por otro lado tendremos un listado con las reglas que permiten modificar y actualizar las distintas variables definidas. Estas reglas constan de:

- **ID:** identificador único de cada regla que permite diferenciarla del resto de reglas definidas.
- **Condition:** la condición que, si se cumple, hará que la variable de usuario se modifique.
- **Action:** acción que se llevará a cabo en caso de que la **Condition** se cumpla, es decir, es el conjunto de modificaciones a realizar a las variables de usuario previamente definidas, en caso de que se cumpla la condición de la regla. podrá contener una o varias fórmulas que determinarán los nuevos valores de una o varias variables de entre las especificadas previamente por el usuario.

En este punto debemos mencionar también qué variables del programa se ha decidido que sean empleables para la modificación de variables y definición de las reglas por parte del usuario y los motivos por los que se ha decidido esto. Las variables disponibles para la definición de las reglas son:

- **N:** número total de células de toda la población (la suma del número de células de todos los genotipos).

- **n_x**: número de células de un genotipo concreto, por ejemplo, **n_A** será el número de células del genotipo con el gen A mutado, y **n_A_B** las del genotipo que tiene mutado los genes A y B.
- **T**: tiempo transcurrido desde el inicio de la simulación, en unidades arbitrarias de tiempo.
- **Variables de usuario**: otras variables de usuario definidas en la misma simulación.

Estas variables se han añadido con la finalidad de dar a las variables de usuario la mayor flexibilidad posible para que el usuario las defina de acuerdo a sus necesidades, considerando todas las variables interesantes tanto para la terapia adaptativa como desde un punto de vista biológico para estudiar su evolución.

Las variables **N**, **n_x** y **T** se consideran necesarias para la terapia adaptativa, ya que permiten definir variables de usuario que se modifiquen de acuerdo a la evolución del tamaño del tumor (**N**), a la cantidad de las células de un determinado tipo, que se consideren más perjudiciales o sensibles a un determinado tratamiento (**n_x**) o al tiempo transcurrido (**T**). Estas variables definidas de acuerdo a estos atributos se podrían emplear luego para definir las intervenciones que se pueden simular con el programa OncoSimulR[6], permitiendo adaptarlas de forma adecuada a las distintas combinaciones posibles de estos tres atributos a lo largo de la simulación, generando así la posibilidad de simular terapia adaptativa. A estas tres variables se les suman las propias variables de usuario, para poder así generar combinaciones aún más complejas que permitan simular escenarios muy complejos (como se ilustrará en la sección 5.3).

Se permite también el uso de otras variables que incluyen **birthRate**, **deathRate** y **mutationRate** que corresponden a las tasas de nacimiento, muerte y mutación de cada genotipo, determinando con qué asiduidad ocurren estos eventos. Estas se han añadido con una finalidad más informativa, ya que, a pesar de no ser tan interesantes desde el punto de vista de la terapia adaptativa, dan una información importante acerca de la adaptabilidad de los distintos genotipos a lo largo de la simulación, haciendo que el estudio de la evolución de variables de usuario que sean combinación de estas variables a lo largo de la simulación pueda tener un gran interés para el estudio tumoral.

3.3. Algoritmo de actualización

El algoritmo de actualización de las variables de usuario es a priori, bastante sencillo, pero su principal complejidad radica en su implementación e integración en el flujo de código principal, como veremos en la sección 4.5.

De forma general el algoritmo de actualización es:

1. El usuario define las variables de usuario que desea y las reglas para modificarlas
2. Se verifica que reglas y variables están definidas correctamente

3. Cada unidad de tiempo muestreada en la simulación:

3.1. Para cada regla:

3.1.1. Si se cumple la Condition:

3.1.1.1. Modificamos el valor de las variables de usuario según lo especificado por el usuario en el atributo Action

3.1.2. Se pasa a la siguiente regla volviendo a 3.1.1

3.2. Se procede a la ejecución de las intervenciones con los valores de las variables calculados

3.4. Definición de la estructura de salida

Para obtener los datos de evolución de las variables de usuario definidas en el tiempo a lo largo de la simulación, se deberán modificar la salida del programa de simulación para que incluya estos. Se debe añadir por tanto a la salida:

- Una lista con los nombres (“ids”) de las variables de usuario creadas.
- Una matriz con los valores de las variables de usuario en cada unidad de tiempo muestreada (en el mismo orden que los nombres de las variables en la lista de nombres de las variables de usuario).

La finalidad de añadir estos datos a la salida de la simulación es poder emplear las variables de usuario (que son arbitrariamente complejas) como datos de salida y no solo como valores a emplear en otros procesos como las intervenciones. Por ejemplo, obtener estos datos permite al usuario graficar la evolución en el tiempo de algunas variables de usuario creadas, lo que puede ser de gran interés para evaluar la evolución del tumor en la simulación. Además, la adición de estos datos en la salida facilitará enormemente las labores de testeo de la funcionalidad implementada. Ejemplos de ambos casos se mostrarán en las secciones 5.3.1 y 5.3.2’.

4

Desarrollo

4.1. Introducción

En esta sección se va a explicar en primer lugar cómo se ha procedido con la unificación de las dos ramas principales con funcionalidad divergente de las que se partía, incluyendo la solución de los errores derivados del merge realizado, así como de errores previos de funcionalidad independientes del merge. Tras esto se procederá a la explicación de la implementación realizada para la funcionalidad definida en el apartado de diseño.

El desarrollo de los ficheros de código que componen esta funcionalidad se ha realizado en los lenguajes R[15] y C++ por ser los lenguajes en los que ya está desarrollado el grueso del programa **OncoSimulR**. R se emplea para la interacción con el usuario, por ser un lenguaje muy común en bioinformática; por su parte C++ está presente para la ejecución del algoritmo de simulación, por ser un lenguaje más eficiente para la ejecución de algoritmos complejos, reduciendo significativamente el coste temporal de ejecución de la simulación. Todo el código desarrollado se puede encontrar en el repositorio de github especificado en las referencias[16].

A lo largo de este apartado se mencionará reiteradamente el uso de las librerías **Rcpp** y **ExprTk**.

- **Rcpp**[17] es un paquete del lenguaje R que permite el paso de datos y objetos entre los lenguajes R y C++, transformando los tipos de dato de uno de los lenguajes a sus equivalentes más próximos en el otro. De este modo, este paquete permite la comunicación fluida entre ambas partes del programa, la parte dedicada a la comunicación con el usuario (desarrollada en R) y la parte computacionalmente más costosa que implementa el grueso del algoritmo de simulación tumoral (desarrollada

en C++).

- **ExprTk**[18] es una librería definida en C++ que permite el análisis de expresiones matemáticas complejas. Esta librería ya estaba siendo empleada en el programa para el análisis de las expresiones que definen algunas funciones de fitness y las intervenciones, pero en la nueva funcionalidad ExprTk será empleada también para analizar las expresiones que permitirán la modificación de las variables de usuario durante la simulación.

4.2. Unificación de Ramas

Cómo se ha explicado en el punto 2.2, al iniciar este trabajo se partía de dos ramas con versiones distintas del programa OncoSimulR[6] cada una con una funcionalidad recientemente implementada en cada una de ellas. Estas ramas debían unificarse para proceder a la implementación de la nueva funcionalidad sobre la versión combinada de ambas.

La unificación de ambas ramas se realizó de manera manual, ya que el merge automático proporcionado por la herramienta Git generaba un gran número de conflictos y no respetaba la funcionalidad de las ramas. Al proceder, se decidió realizar la fusión sobre la rama que contenía la funcionalidad de las tasas de nacimiento y muerte (explicada en el punto 2.2.1) por ser la que más modificaciones introducía sobre el código base de la simulación, y sobre esta se introdujeron los cambios correspondientes a la rama con la funcionalidad de las intervenciones (explicada en el punto 2.2.2).

Para realizar la unificación de ambas ramas se procedió del siguiente modo:

1. Sobre la rama que se ha indicado anteriormente se añadieron los ficheros de código R y C++ exclusivos de la funcionalidad de intervenciones (`intervention.cpp`, `intervention.h`, `intervention.R`, `multivariorant_hyergeometric.cpp` y `multivariorant_hyergeometric.h`), los ficheros de testeo de esta funcionalidad (`test.intervention.R` y `test.Z-intervention.R`) y los ficheros de documentación y viñetas que explican e ilustran el funcionamiento de esta.
2. Tras esto se procedió a la modificación de la función **updatePopulations** del fichero `intervention.cpp`, encargada de actualizar las poblaciones de los distintos genotipos tras la ejecución de una intervención, para que empleasen las tasas de nacimiento y muerte implementadas en la funcionalidad de la otra rama.
3. Se actualizó el flujo de código principal del algoritmo BNB de Mather[11] (explicado en el punto 2.1) en el fichero `BNB_nr.cpp` para añadir las llamadas necesarias para añadir la funcionalidad correspondiente a la ejecución de las intervenciones.
4. Por último se modificaron las cabeceras y llamadas a todas las funciones del programa OncoSimulR para que tomasen de forma adecuada el argumento `interventions`, teniendo especial cuidado al tratar con los ficheros `RcppExports.R`,

RcppExports.cpp y OncoSimulR_init.c, encargados de la comunicación entre los ficheros en R y en C++ mediante la librería Rcpp.

Posteriormente se realizó un merge automático con una tercera rama modificada por el tutor de este trabajo para incluir unas pequeñas modificaciones en la documentación del programa.

Al finalizar este proceso, se procedió a la comprobación de que la funcionalidad de ambas ramas era correcta en la rama combinada mediante la ejecución de los tests pertinentes, y a asegurar que el merge no había afectado al resto de características del programa. Todo este testeo se explicará de forma más detallada en el punto 5.1.

4.3. Optimización del código de intervenciones

Durante la realización del merge de las ramas previas, se encontraron algunos problemas de optimización en el código de las intervenciones que hacían que el tiempo de ejecución fuese más alto de lo deseado. Por tanto antes de proseguir con la nueva funcionalidad se procedió a solucionar esto.

El principal problema detectado en este aspecto era la existencia de fragmentos de código que se ejecutaban en todas las iteraciones del algoritmo de simulación, independientemente de si se realizaban intervenciones o no, cuando solo era necesaria su ejecución cuando estas sí se realizaban. Si bien estos fragmentos de código no eran en sí mismos muy costosos computacionalmente, en una simulación de gran tamaño, con un gran número de iteraciones (como las que se realizan habitualmente en este programa), estos fragmentos se estarían ejecutando de forma innecesaria un número enorme de veces, haciendo que el incremento en el tiempo de ejecución sí fuese bastante significativa.

Los cambios realizados, por tanto, fueron:

- Modificación de la función **executeInterventions** de interventions.cpp:
 - Cambiamos la función para que devuelva “true” si ha habido una intervención y “false” en caso contrario en lugar de devolver “void” como hasta ahora. Esto nos permite, desde el algoritmo principal, saber si ha habido o no intervención para ejecutar los fragmentos de código necesarios; además nos permite desde fuera obtener los tiempos en que se ejecutan las intervenciones, facilitándonos el testeo posteriormente.
 - Hacemos que en la función solamente se llame a **updatePopulations** (que actualiza las poblaciones de cada genotipo) en los casos en los que se ha ejecutado alguna intervención, puesto que son los únicos casos en que las poblaciones pueden haber cambiado.
- Modificamos también la función **parseWhatHappens** para que devuelva “void” en vez de “true/false”. Antes devolvía “true” si se ejecutaba sin errores y en caso de

error lanzaba una excepción y devolvía “false”, lo cual no tenía sentido. Por tanto ahora no devuelve nada y en caso de error simplemente lanza una excepción.

Tras esto de nuevo procedemos a comprobar que la funcionalidad es correcta mediante los tests explicados en el punto 5.1, y comenzamos la implementación de la nueva funcionalidad.

4.4. Implementación de variables de usuario y reglas

4.4.1. En R

Desde R debemos hacer posible que el usuario pueda definir las variables de usuario que desee, y las reglas que quiere que se empleen para su cálculo y actualización a lo largo de la simulación, así como permitir que pase estas como argumento a la función **OncoSimulIndiv** de OncoSimulR, que es la función responsable de llevar a cabo la simulación.

Para ello en primer lugar vemos como se deben definir las variables de usuario y las reglas en R:

- **Variables de usuario:** las variables de usuario hemos visto en el punto 3.2 que simplemente contienen un nombre que identifica a la variable, y un valor numérico. Por tanto, el usuario definirá en R una simple lista con todas las variables de usuario que desee crear para la simulación con estos atributos, del siguiente modo:

```
userVars <- list(  
  list(Name = "user_var_1",  
        Value = 0  
  ),  
  list(Name = "user_var_2",  
        Value = 0  
  ),  
  list(Name = "user_var_3",  
        Value = 0  
  )  
)
```

- Name: string que contiene el nombre que identificará a la variable; debe ser único para la variable.
 - Value: valor inicial que tomará la variable al inicio de la simulación.
- **Reglas:** del mismo modo hemos visto en 3.2 que las reglas constan de un ID, un Condition y una Action. De igual forma el usuario definirá estas reglas mediante una lista en R como en el siguiente ejemplo:

```

rules <- list(
  list(ID = "rule_1",
       Condition = "T > 20",
       Action = "user_var_1 = 1"
  ), list(ID = "rule_2",
         Condition = "T > 30",
         Action = "user_var_2 = 2"
  ), list(ID = "rule_3",
         Condition = "T > 40",
         Action = "user_var_3 = 3; user_var_2 = 1"
  )
)

```

- **ID:** string identifica la regla y la distingue del resto, debe ser único para cada regla.
- **Condition:** expresión con valor true/false que determina si se ejecuta o no la regla. Puede emplear las variables especificadas en el punto 3.2.
- **Action:** una o varias expresiones con valor numérico separadas por “;” que determinan el valor que tomarán las variables de usuario en caso de que se ejecute la regla. Del mismo modo que el campo “Condition”, puede emplear las variables especificadas en el punto 3.2.

Partiendo de estas definiciones, creamos en R una serie de funciones que comprueban que el usuario ha especificado las variables de usuario y reglas de forma correcta y las adaptan para que puedan pasarse a C++ de forma correcta mediante la librería Rcpp[17]. Estas funciones son, en el caso de las variables de usuario:

- **check_same_name:** comprueba que en la lista definida no existen varias variables de usuario con el mismo nombre.
- **verify_user_vars:** comprueba que todas las variables de usuario definidas en la lista tienen todos los atributos necesarios (Name y Value).
- **createUserVars:** mediante llamadas a las dos funciones anteriores comprueba que la lista de variables de usuario definida es correcta y, si lo es, la devuelve.

En el caso de las reglas:

- **check_double_rule_id:** comprueba que en la lista de reglas definida no hay dos reglas con el mismo id.
- **check_action:** comprueba la correcta especificación de la expresión del campo Action. Para ello comprueba que, si hay varias expresiones, estas están correctamente separadas por “;” y que cada una de las expresiones tiene la forma: <variable de usuario> = <expresión>.

- **verify_rules**: comprueba que las reglas contienen todos los atributos necesarios (id, Action y Condition), y que estos son correctos llamando a las dos funciones anteriores.
- **transform_rule**: modifica las expresiones Action y Condition de las reglas, ya que aunque en estas las variables **n_X** (explicadas en 3.2) se especifican como **n_A**, **n_B**, etc; en C++ los genotipos se definen con números y no con letras, por tanto esta función transforma **n_A**, **n_B**... en **n_1**, **n_2**...
- **adapt_rules_to_cpp**: mediante llamadas a la función anterior, modifica las expresiones Action y Condition de las distintas reglas definidas en la lista para adaptarlas a C++.
- **createRules**: mediante llamadas a **verify_rules** y a **adapt_rules_to_cpp** comprueba que las reglas definidas son correctas y las adapta a C++ devolviendo la lista de reglas ya transformada.

La implementación de estas funciones puede encontrarse en https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/R/user_var.R.

Por último, en R, se han modificado las llamadas y la cabecera de la función `OncoSimulIndiv` y de las demás funciones internas de `OncoSimulR`[6] para que reciban de forma adecuada los nuevos argumentos *userVars* (lista de variables de usuario) y *rules* (listado de reglas).

4.4.2. En C++

Ya en C++ procedemos a desarrollar el tipo abstracto de datos (TAD) que maneja las variables de usuario y reglas definidas por el usuario en R. Para ello en primer lugar definimos las estructuras que contienen estos objetos. Para las reglas creamos la estructura **Rule** en C++, del siguiente modo:

```
typedef struct {
    std::string id; // identifier of the rule
    std::string condition; // condition that must be true for the
        ↪ rule to execute
    std::string action; // action to be performed when the condition
        ↪ is satisfied
} Rule;
```

Donde todos los campos son strings que corresponden exactamente a los atributos especificados en R por el usuario, explicados en el punto anterior 4.4.1.

Por otro lado definimos otra estructura en C++ llamada **UserVarsInfo**, que contiene las variables de usuario, y otros datos necesarios para operar adecuadamente con ellas. Esta estructura es:

```
typedef struct{
    std::map<std::string, double> userVars; // maps a user variable
        ⇨ name to its current value
    std::vector<Rule> rules; // rules that apply to the group of user
        ⇨ variables
    std::map<std::string, double> mapGenoToPop; // variable that maps
        ⇨ a genotype to its population
} UserVarsInfo;
```

Donde:

- **userVars** es un mapa cuyas claves son strings con los nombres de las variables de usuario definidas y los valores son números reales que definen el valor de las distintas variables.
- **rules** es un listado con las reglas que controlan la actualización de las variables, definidas como estructuras Rule definidas justo antes.
- **mapGenoToPop** es un mapa que contiene las poblaciones de todos los genotipos en un momento dado de la simulación. Es necesario para poder operar con estos valores al actualizar las variables de usuario empleando las reglas.

Una vez tenemos estas estructuras definidas, procedemos a la elaboración de las distintas funciones que implementan la funcionalidad del TAD. Comenzando por algunas funciones muy sencillas que crean, destruyen, comparan, imprimen e implementan alguna funcionalidad de estas estructuras:

- **createRule**: función que rellena la estructura Rule.
- **createUserVarsInfo**: función que rellena la estructura UserVarsInfo.
- **addUserVar**: función que añade una variable de usuario al mapa de variables de usuario de UserVarsInfo.
- **addRule**: función que añade una regla al listado de reglas de UserVarsInfo.
- **compareRules**: función que compara dos reglas definidas por la estructura Rule.
- **destroyRule**: función que elimina una regla del listado de reglas de la estructura UserVarsInfo.
- **printRule**: función que imprime por pantalla una regla definida por la estructura Rule.
- **printUserVarsInfo**: función que imprime por pantalla las variables de usuario y reglas contenidas en una variable del tipo UserVarsInfo.

- **isValid**: función que comprueba que cada una de las subexpresiones separadas por “;” contenidas en el atributo Action de la estructura Rule están bien definidas, con la forma `<userVar>= <expresión>`.

Las cabeceras de estas funciones pueden encontrarse en el apéndice C, y su implementación detallada es visible en el link https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/src/user_var.cpp.

Por último definimos las funciones que implementan la funcionalidad principal del TAD, estas son **executeRules** y **parseAction** que, en conjunto, permiten la actualización de los valores de las variables de usuario siguiendo lo especificado en las reglas.

- **executeRules**:

```
void executeRules (UserVarsInfo& uvif ,
                  double &currentTime ,
                  const std::map<std::string , double>
                    ↪ birthMap ,
                  const std::map<std::string , double>
                    ↪ deathMap ,
                  const std::map<std::string , double>
                    ↪ mutationMap);
```

Esta es la función principal que implementa el algoritmo de actualización de las variables de usuario explicado en el apartado 3.2. Para ello hace uso de la librería **ExprTk**[18] para analizar la expresión booleana contenida en el campo **Condition** y determinar si una regla debe ejecutarse o no, y, en caso afirmativo llama la segunda función mencionada, **parseAction**, para ejecutar la modificación de las variables pertinentes. Los parámetros que recibe esta función son:

- **uvif**: estructura **UserVarsInfo** (explicada al inicio de este punto) que contiene el listado de variables de usuario y reglas definidas para la simulación en ejecución. Se emplea para acceder a las reglas y variables de usuario, así como para añadir las variables de usuario a la tabla de símbolos de la librería **ExprTk** para poder emplearlas en el parseo de las expresiones contenidas en las reglas. Contiene además también el campo **mapGenoToPop** con las poblaciones asociadas a cada genotipo, que se deben añadir también a la tabla de símbolos (corresponden a las variables **n_x**) y son necesarias para calcular la población total **N**.
- **currentTime**: tiempo actual de la simulación en el ciclo en que se está ejecutando la función. Se emplea para añadirlo como variable a la tabla de símbolos de **ExprTk** para emplearse en el parseo de expresiones. Corresponde a la variable **T** explicada en el punto 3.2.
- **birthMap**, **deathMap** y **mutationMap**: mapas que contiene las tasas de nacimiento, muerte y mutación de todos los genotipos presentes en el momento actual de la simulación. Se añaden como variables a la tabla de

símbolos de ExprTk. Corresponden a las variables **birthRate**, **deathRate** y **mutationrate** respectivamente explicadas en el punto 3.2.

Conociendo estos datos, la función opera de la siguiente manera:

Algoritmo 1 executeRules

Rellenamos la tabla de símbolos de ExprTk con las variables mencionadas en el apartado 3.2 para que la librería pueda operar con ellas al analizar la **Condition**.

for regla **in** uvif.rules **do**

Analizamos **regla.Condition** con ExprTk obteniendo **true/false**

if regla.Condition == true **then**

Llamamos a **parseAction**

end if

end for

▪ **parseAction:**

```
void parseAction(UserVarsInfo& uvif ,
                Rule rule ,
                double currentTime ,
                const std::map<std::string , double>
                    ↪ birthMap ,
                const std::map<std::string , double>
                    ↪ deathMap ,
                const std::map<std::string , double>
                    ↪ mutationMap);
```

Esta función se encarga de parsear el campo **Action** de la regla mediante ExprTk[18] y modificar el listado de variables de usuario definidas según lo indicado en este campo. En este caso los parámetros que recibe son muy similares a los de la función anterior **executeRules**, pero añadiendo **rule**, que contiene la regla concreta que se va a ejecutar. Esta segunda función tiene la siguiente estructura:

Algoritmo 2 executeRules

Rellenamos la tabla de símbolos de ExprTk con las variables mencionadas en el apartado 3.2 para que la librería pueda operar con ellas al analizar la **Action**.

Dividimos la string **rule.action** en **tokens** separados por “;” (cada token define la actualización de una variable)

for each token **do**

Separamos la string que define la asignación por el “=” obteniendo el **nombreVariable** y la **expresión**

Analizamos la **expresión** con ExprTk obteniendo el **valor** numérico al que equivale
uvif.userVars[**nombreVariable**] = **valor**

end for

Es importante tener en cuenta que las reglas de actualización de las variables de usuario se ejecutan en orden, y es responsabilidad del usuario tener en cuenta las implicaciones que esto conlleva. Por ejemplo, supongamos las reglas:

- ID: rule1; Condition: $n_A > 100 \text{ AND } user_var_1 = 0$; Action: $user_var_1 = 1$.
- ID: rule2; Condition: $user_var_1 = 0$; Action: $user_var_1 = 2$.

En este caso, si tenemos $n_A = 110$ y $user_var_1 = 0$, se cumplirá la condición de la regla *rule1*, y esta se ejecutará haciendo que $user_var_1 = 1$. Por tanto, cuando se vaya a comprobar la condición de la regla *rule2*, la variable $user_var_1 = 1 \neq 0$ por lo que esta segunda regla no se ejecutará, entonces $user_var_1$ terminará valiendo 1.

4.5. Integración en el flujo principal de código

Una vez definidas la funcionalidad de las variables de usuario y las reglas en el TAD anteriormente explicado, se procede a la integración de esta en el flujo principal de código de la simulación tumoral. Para ello se han incluido fragmentos de código que ejecutan las funciones de actualización de las variables de usuario dentro de la función **nr_innerBNB** del fichero **BNB_nr.cpp**, que es la que implementa el algoritmo BNB de Mather[11] explicado en el punto 2.1.

La actualización de las variables de usuario se incluye dentro de esta función, de manera que se ejecute una vez por cada ciclo de la simulación. Con el fin de poder simular la terapia adaptativa, es necesario poder emplear estas variables de usuario calculadas en la ejecución de las intervenciones. Esto se debe a que la idea general del flujo de código es:

1. Actualizar tamaños de población de cada genotipo (y población total) según el algoritmo de simulación.
2. Actualizar las tasas de nacimiento y muerte de cada genotipo si es necesario.
3. Evaluar el valor de las variables de usuario y actualizarlos siguiendo las reglas definidas para ello.
4. Ejecutar las intervenciones que pueden o no depender de estas variables. La ejecución de las intervenciones afectará a las variables de usuario y a otras variables tales como la población total, población de cada genotipo, frecuencias relativas, etc.

Por tanto, esta actualización debe realizarse antes de que las intervenciones se ejecuten en cada ciclo. Finalmente se ha decidido realizar la actualización justo en el punto entre la actualización de las poblaciones de los genotipos según las tasas de nacimiento, muerte y mutación, y la ejecución de las intervenciones; de este modo las variables de usuario se actualizan de acuerdo a los datos temporalmente más cercanos posibles a la ejecución de las intervenciones que es, la principal funcionalidad para la que se van a emplear estas variables.

Se valoró la posibilidad de recalcular de nuevo las variables de usuario después de la ejecución de las intervenciones, pues estas pueden variar ciertos datos tales como la

población, que afectarían al valor de estas variables. Sin embargo, esta opción se terminó descartando puesto que no tendría sentido siguiendo la lógica de la simulación. En primer lugar, dado que el algoritmo funciona de acuerdo a unos ciclos de actualización, actualizar las variables más de una vez por ciclo supondría una incongruencia con respecto al resto del algoritmo. Además podría dar lugar a problemas como bucles infinitos, pues al actualizar de nuevo estas variables, podría “activarse” el “trigger” de una intervención que previamente no se había activado, por lo que induciría a que se deberían ejecutar de nuevo las intervenciones, dando situaciones del tipo:

1. Se modifican las poblaciones de forma natural por el algoritmo de simulación.
2. Se calculan las variables de usuario.
3. Se calculan las intervenciones que modifican poblaciones.
4. Recalculamos las variables de usuario puesto que han variado las poblaciones
5. Al cambiar los valores de las variables de usuario se activan nuevos “triggers” de las intervenciones, luego volvemos a 3.

Para realizar la actualización de las variables en el algoritmo de simulación principal mencionado, añadimos a este el siguiente fragmento de código:

```
std::map<std::string, double> birthMap = evalFVarsFitness(
    ⇨ fitnessEffects, Genotypes, popParams, "birth");
std::map<std::string, double> deathMap = evalFVarsFitness(
    ⇨ fitnessEffects, Genotypes, popParams, "death");
std::map<std::string, double> mutationMap = evalFVarsFitness(
    ⇨ fitnessEffects, Genotypes, popParams, "mutation");
if(rules_length > 0 && userVars_length > 0){
    uvif.mapGenoToPop = evalFVars(fitnessEffects, Genotypes,
        ⇨ popParams, true);
    executeRules(uvif, currentTime, birthMap, deathMap,
        ⇨ mutationMap);
    auxValues = {};
    for (auto &item : uvif.userVars){
        auxValues.push_back(item.second);
    }
    auxValues.push_back(currentTime);
    userVarValues.push_back(auxValues);
}
```

En este fragmento cabe destacar la creación y uso de la función **evalFVarsFitness** que permite acceder a las tasas de nacimiento, muerte y mutación de los distintos genotipos presentes en este momento de la simulación. Estas variables se deben pasar como parámetros a **executeRules** para poder añadirse a la tabla de símbolos de ExprTk[18]. Vemos que de forma similar se emplea también la función **evalFVars**, que ya existía con anterioridad, para obtener los valores de las poblaciones de los distintos genotipos,

valores que también se deben pasar como parámetro a **executeRules**. Una vez obtenidos todos estos valores necesarios, se procede a la llamada a **executeRules** para actualizar las variables, almacenando los valores de las variables resultantes en vectores auxiliares para posteriormente poder devolverlos a la salida.

Además de estas adiciones al algoritmo, se ha modificado también su estructura de salida para que incluya una lista con los nombres de las variables de usuario creadas durante la simulación, y una matriz con los valores que toman las variables en cada unidad de tiempo de la simulación (cada fila es una lista con los valores de las distintas variables para una unidad de tiempo, en el mismo orden en que aparecen en la lista con los nombres de las variables). Esta matriz se rellena con los valores que se han ido almacenando en el vector auxiliar mencionado anteriormente.

Por último, para finalizar esta integración en el flujo principal de código, se han modificado los parámetros de entrada de todas las funciones en R de OncoSimulR[6] para que reciban de forma adecuada los parámetros **userVars** y **rules**, y se han actualizado los ficheros RcppExports.R, RcppExports.cpp y OncoSimulR_init.c, para que estos mismos parámetros se pasen a C++ de forma adecuada.

4.6. Variables de usuario en las intervenciones. Terapia adaptativa

Como hemos visto en la sección 3.1, la finalidad principal de la creación de las variables de usuario es lograr la simulación de la terapia adaptativa. Para ello, ahora que contamos con la funcionalidad completa de las variables de usuario, es preciso que estas puedan emplearse para definir las expresiones que determinan las acciones a realizar en una intervención.

Por tanto, se ha incluido estas variables en la funcionalidad de las intervenciones. Para ello se han modificado las funciones **executeInterventions** y **parseWhatHappens** del fichero **interventions.cpp**, que controlan la ejecución de las intervenciones en la simulación. Ambas funciones reciben unas modificaciones muy similares que consisten en:

1. Modificamos ambas funciones para que para que reciban como argumento la estructura **userVarsInfo** con los datos de las variables de usuario.
2. Añadimos en ambas un pequeño fragmento de código para introducir las variables de usuario y sus valores a la tabla de símbolos de ExprTk[18] permitiendo así que se puedan emplear en el análisis de expresiones de las intervenciones.
3. Modificamos las cabeceras y llamadas a estas dos funciones de forma adecuada para que incluyan **userVarsInfo** en todas las llamadas.

Con estos cambios se finaliza el desarrollo de la funcionalidad. En el siguiente capítulo se detallan las pruebas para comprobar su correcto funcionamiento.

5

Pruebas y Resultados

5.1. Comprobación de funcionalidad previa

Antes de proceder a introducir toda la nueva funcionalidad explicada a partir del punto 4.4, y tras realizar la unificación de las dos ramas de las que se partía y la solución de los problemas de optimización explicados en 4.3, aseguramos que la funcionalidad anterior no se ha visto afectada mediante el uso de la batería de tests ya disponibles para el programa **OncoSimulR**[6]. Esta batería de tests incluye por una lado unos tests manuales muy extensos y costosos computacionalmente¹, y otro grupo de tests que comprueban la funcionalidad básica del programa mediante el paquete **testthat**[19] del lenguaje R². Al ejecutar estos tests se detectaron una serie de errores previos al merge en una de las ramas. Los errores detectados están localizados en los tests de la funcionalidad de intervenciones, y son errores en los propios tests y no fallos de funcionalidad. Para solucionar estos errores:

- Se modifica la salida del algoritmo de simulación BNB implementado en la función **nr_BNB_Algo5** para que incluya una lista con los tiempos de la simulación en los que se ejecutan intervenciones (esto nos permite buscar los puntos exactos donde se realizan las intervenciones en los tests). De este modo la salida de la función principal de simulación **OncoSimulIndiv** ahora incluye el campo **x\$other\$interventionTimes** con estos tiempos (donde “x” es la variable donde se almacena la salida de la función).
- Adelantamos el punto de ejecución de las intervenciones en el flujo principal del código de la simulación para que se realicen antes de rellenar las estructuras

¹<https://github.com/JavierLopezC/OncoSimul/tree/adaptive-ther/OncoSimulR/tests/manual>

²<https://github.com/JavierLopezC/OncoSimul/tree/adaptive-ther/OncoSimulR/tests/testthat>

de salida. Con esto conseguimos que al actualizarse las poblaciones y tasas de nacimiento y muerte al final de cada ciclo de simulación, estas incluyan los cambios realizados por las intervenciones. Antes de esta modificación, los cambios que realizaban las intervenciones no se veían reflejados en las poblaciones mostradas a la salida hasta el tiempo correspondiente a la siguiente iteración de la simulación, lo que generaba un desfase entre lo que se testeaba (que emplea los datos de salida) y las poblaciones internas empleadas en la simulación (que sí incluían los cambios realizados por las intervenciones).

- Comentar la comprobación de `totPopSize != totalPop` en `reduceTotalPopulation` (`interventions.cpp`) porque al haberlo adelantado `totPopSize` no está actualizado.
- Añadimos un fragmento de código para eliminar los genotipos con población cero después de realizar intervenciones. Las poblaciones de tamaño cero se deben eliminar de las tablas pues estas solo contienen poblaciones existentes.
- Modificamos los tests ya existentes de intervenciones para adaptarlos a estos nuevos cambios y comprobamos que todos los tests se pasan correctamente. Estas modificaciones consisten principalmente en hacer que se empleen los tiempos almacenados en `x$other$interventionTimes` para buscar los puntos en los que se realizan intervenciones y comprobar que estas son correctas, en lugar de buscarlas recorriendo la estructura de salida con las poblaciones en cada unidad de tiempo, pues este método no permitía un testeo correcto además de ser mucho menos eficiente.

Por último, tras implementar toda la nueva funcionalidad (explicada a partir del punto 4.4) volvemos a ejecutar toda la batería de tests para asegurar, de nuevo, que estos cambios no han afectado al correcto funcionamiento de la funcionalidad previa. Nótese que cada vez que se ejecuta el comando “R CMD check” se ejecutan todos los tests de la herramienta `Testthat`[19], siendo la forma habitual de operar ejecutar en orden:

1. R CMD build
2. R CMD check

5.2. Tests de nueva funcionalidad

Para probar el correcto funcionamiento de la nueva funcionalidad implementada que incluye la creación de variables de usuario y su actualización, así como la simulación de la terapia adaptativa mediante la inclusión de estas variables en las intervenciones, se ha ampliado la batería de tests del programa. Se han añadido tanto tests que emplean la herramienta `testthat`[19] incluyendo un nuevo documento de tests `testthat\test.user_var.R`, y los tests manuales añadiendo el fichero `manual\user_vars\test.user_var.R` que, en conjunto, prueban la funcionalidad

de las variables de usuario. Se ha ampliado también el documento de tests **testthat\test.intervention.R** que incluye los tests sobre intervenciones, para añadir los casos en que estas dependen de los valores contenidos en variables de usuario. Todos estos están incluidos en el directorio **tests**³.

Además de los tests, se ha elaborado también la documentación de las funciones más relevantes (ver apéndice B) y se han ampliado las viñetas del programa con ejemplos ilustrativos de la nueva funcionalidad (ver apéndice A).

5.2.1. Variables de usuario

Cómo hemos comentado, los tests que comprueban el correcto funcionamiento de las variables de usuario están contenidos en los ficheros **test.user_var.R** de las carpetas **tests\testthat** y **tests>manual\user_vars**. Estos tests contienen las siguientes comprobaciones:

- Las variables de usuario se crean correctamente.
- Dos variables de usuario no pueden tener el mismo nombre.
- Las reglas para la actualización de las variables de usuario se crean correctamente.
- Dos reglas no pueden tener el mismo ID.
- Al ejecutarse las reglas, las variables de usuario se modifican de forma correcta (en función de la variable T).
- Al ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la variable N).
- Al ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de las poblaciones por genotipo n_x).
- Al ejecutarse las reglas las variables de usuario se modifican de forma correcta (cuando la *Condition* o *Action* dependen de otras variables de usuario definidas previamente).

5.2.2. Intervenciones con variables de usuario

Una vez hemos comprobado que las variables de usuario funcionan de forma adecuada, ampliamos los tests del fichero **test.intervention.R** para asegurar que las intervenciones funcionan de forma correcta cuando depende de las variables de usuario creadas. Para ello añadimos dos nuevos tests en este fichero para comprobar:

³<https://github.com/JavierLopezC/OncoSimul/tree/adaptive-ther/OncoSimulR/tests>

- Las intervenciones se ejecutan de forma correcta cuando el **Trigger** (condición de ejecución de la intervención) depende de variables de usuario.
- Las intervenciones se ejecutan de forma correcta cuando el **WhatHappens** (modificación en poblaciones o tasas realizada por la intervención) depende de variables de usuario.

5.3. Resultados

En esta sección vamos a comprobar que, efectivamente, los resultados obtenidos cumplen con lo esperado e, incluso, se han logrado resultados positivos que no se habían planteado inicialmente como objetivos del trabajo.

5.3.1. Variables de usuario como datos de interés a la salida

Al implementar las variables de usuario, como hemos comentado anteriormente, se permite obtener un enorme número de datos arbitrariamente complejos de la simulación a la salida. Estos pueden ser de gran interés para una mejor comprensión de lo ocurrido durante la simulación. En esta sección se van a plantear algunos ejemplos de ejecución en los que se saca partido a esta funcionalidad.

Representación de la proporción de cada genotipo sobre la población total

Comenzamos con un ejemplo muy sencillo en el que generamos un grupo de variables de usuario que almacenan la proporción de cada genotipo existente sobre el total de la población, es decir, matemáticamente cada una de estas variables es, para cada genotipo “X”:

$$proporcionX = n_X/N$$

Dónde “n_X” es el tamaño de población correspondiente al genotipo X (o sea, el número de células del genotipo X), y “N” la población total (el número total de células).

Con estas variables de usuario definidas iniciamos una simulación con cinco genotipos: **WT** (el inicial, sin mutaciones), **A**, **B**, **A,B** y **A,C**. Asignamos a cada uno de ellos sus fitness y sus poblaciones iniciales (estos detalles se pueden ver en el código adjunto en el apéndice A.1 para ejecutar esta simulación), e iniciamos la simulación llamando a **OncoSimulIndiv**. En este caso simularemos tan solo un tiempo de diez unidades, pues es suficiente para ver los datos que nos proporcionan las variables de usuario.

Ahora representamos gráficamente los resultados obtenidos en las poblaciones de los genotipos y en las variables de usuario (figuras 5.1 y 5.2).

Con sólo los datos de la evolución poblacional (fig. 5.1) podemos observar en líneas generales el avance de las poblaciones de los genotipos, pero no podemos sacar conclusiones

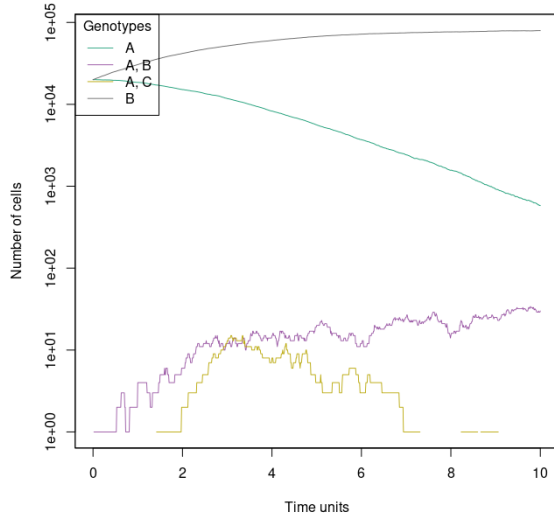


Figura 5.1: Evolución poblacional. Primer ejemplo variables de usuario

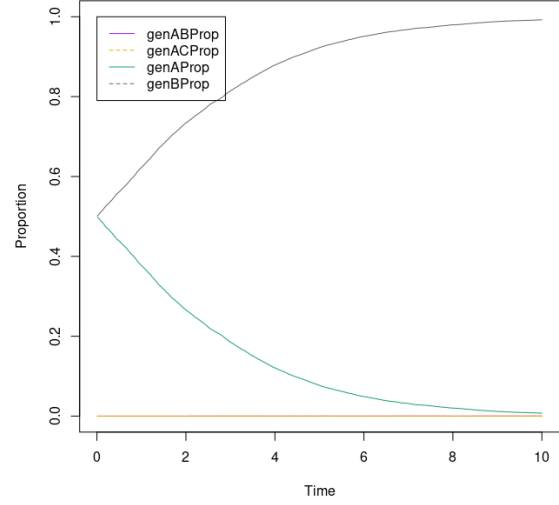


Figura 5.2: Proporción de genotipos en la población. Primer ejemplo variables de usuario

fácilmente acerca de las relaciones entre las poblaciones de los distintos genotipos. (Téngase en cuenta que el gráfico se encuentra en escala logarítmica, por lo que las variaciones en los genotipos **AB** y **AC** son mínimas aunque sean apreciables en el gráfico).

Cuando observamos también los datos de las variables de usuario (fig. 5.2) podemos observar claramente que los genotipos **AB** y **AC** son apenas relevantes comparados con los otros, y además se aprecia que el genotipo **B** aumenta de forma proporcional al descenso en la población del genotipo **A**.

Representación de la diferencia entre la tasa de nacimiento y muerte

Vamos ahora con otro ejemplo para analizar más profundamente el uso de variables de usuario. En este caso las que vamos a definir son variables que contengan el valor de la diferencia entre la tasa de nacimiento y muerte de los distintos genotipos. Este valor, por tanto, nos mostrará si se debería esperar que la población de un determinado genotipo crezca (si la diferencia es positiva) o disminuya (si es negativa). La fórmula para calcular estas variables de usuario es muy sencilla:

$$diferenciaTasasX = b_X - d_X$$

Dónde “b_X” es la tasa de nacimiento correspondiente al genotipo X, y “d_X” la de muerte. Del mismo modo que en el ejemplo anterior, establecemos los datos de la simulación (visibles en el apéndice A.2 en el código para reproducir el ejemplo) que en este caso contará con 3 genotipos **WT**, **A** y **B**, y, para que los datos de las tasas sean variables y más interesantes de analizar, definiremos el fitness como dependiente de las frecuencias de los genotipos, funcionalidad que también permite la herramienta **OncoSimulR**[6].

Representamos los resultados obtenidos para las variables de usuario que representan la diferencia de tasas y la evolución poblacional (figuras 5.3 y 5.4).

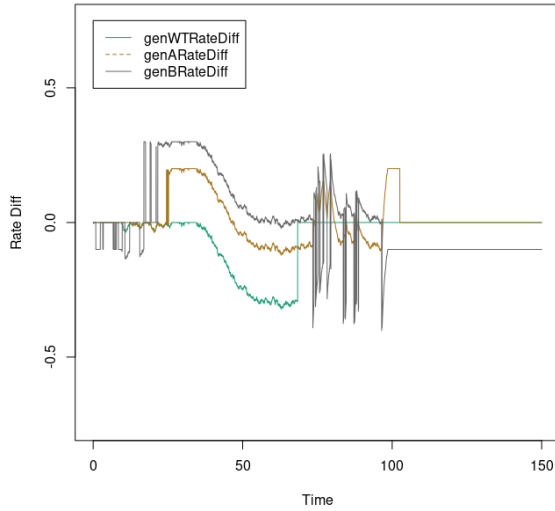


Figura 5.3: Diferencia tasas de nacimiento y muerte. Segundo ejemplo variables de usuario

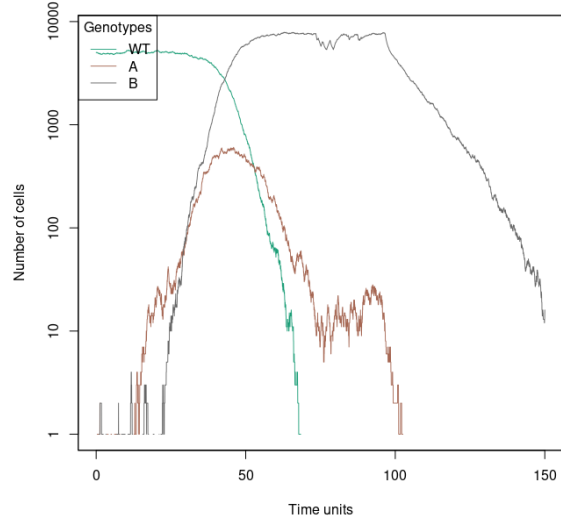


Figura 5.4: Evolución poblacional. Segundo ejemplo variables de usuario

Viendo la gráfica de variables de usuario (fig. 5.3) esto podemos estimar que a partir del tiempo 20-25 los genotipos **A** y **B** comenzarán a crecer, el segundo más rápidamente que el primero. **A** debería comenzar a decrecer alrededor del tiempo 50, cuando la diferencia de tasas pasa a ser negativa, mientras que **B** lo hará más tarde en el tiempo 100 cuando su diferencia de tasas se estabiliza por debajo de cero. Por su parte esperamos que **WT** comience a decrecer alrededor del tiempo 40.

Analizando la evolución poblacional (fig. 5.4) comprobamos que, los resultados son los esperados; las pequeñas variaciones se deben al componente aleatorio que existe en la evolución de las poblaciones. Por tanto, la observación de la diferencia de tasas que nos proporcionan las variables de usuario nos permite ver unos datos objetivos para predecir la evolución poblacional minimizando el efecto de la estocasticidad en el algoritmo.

5.3.2. Terapia Adaptativa

Procedemos ahora a analizar el resultado de la funcionalidad correspondiente al objetivo principal de este trabajo, que es lograr simular la terapia adaptativa. Para ello empleamos una serie de ejemplos de ejecución que nos permitirán ver el funcionamiento de esta.

Selección de tratamiento según el genotipo más adaptado

Comenzamos con un ejemplo en el que simulamos un tratamiento en un paciente con cáncer. En este caso en el tumor aparecen tres genotipos, el genotipo original **WT** sin genes mutados, y los genotipos **A** y **B** con los genes A y B mutados respectivamente. En este ejemplo simularemos un tratamiento que varía para enfocarse en el genotipo con más velocidad de crecimiento y que, por tanto más posibilidades tiene de desarrollarse. Para ello partimos del mismo ejemplo que en el punto anterior (5.3.1) en el que calculamos la diferencia de las tasas de nacimiento y muerte, para de esta forma estimar cual es el genotipo con más posibilidades de proliferar. Además añadimos un mínimo poblacional de 1000 células para detectar determinado genotipo, es decir, que si la población de un genotipo no es superior a 1000 no se tratará, pues no se considera suficiente para ser detectado.

Tras esto definimos una serie de intervenciones para minimizar el desarrollo del genotipo que tenga dicho valor más alto. Estas intervenciones las definimos del siguiente modo:

```
interventions <- list(
  list(ID = "i1",
    Trigger = "genADiff > genBDiff and genADiff > genWTDiff"
    ⇨ " ",
    WhatHappens = "n_A = n_A*0.5",
    Periodicity = 10,
    Repetitions = Inf
  ), list(ID = "i2",
    Trigger = "genBDiff > genADiff and genBDiff > genWTDiff"
    ⇨ " ",
    WhatHappens = "n_B = n_B*0.5",
    Periodicity = 10,
    Repetitions = Inf
  )
)
```

Posteriormente iniciamos la simulación (el código para reproducir este ejemplo está en el apéndice A.3), obteniendo los resultados visibles en las figuras 5.5 y 5.6.

Vemos que efectivamente las bajadas bruscas en la población que corresponden a las intervenciones se producen al comienzo en **WT** que comienza siendo el más adaptado pues tiene mayor diferencia entre las tasas, pero a partir del tiempo 30 aproximadamente, estas pasan a realizarse sobre el genotipo **B**, que justo es a partir de ese punto cuando comienza a tener una diferencia entre tasas mucho más alta.

Simulación de situación verosímil

Por último vamos a simular un escenario complejo de terapia adaptativa intentando acercarnos lo máximo posible a lo que podría ocurrir en un caso real. Para ello

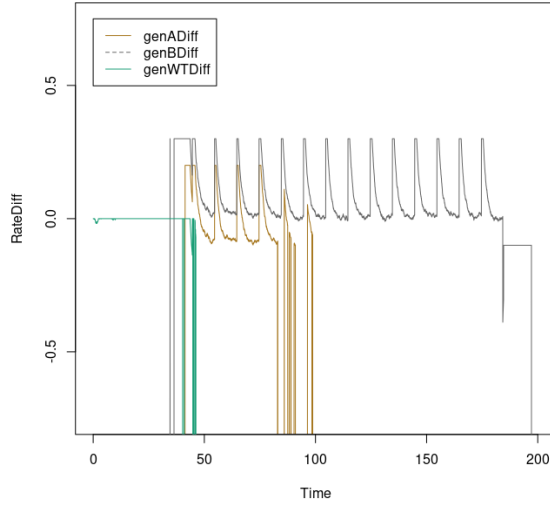


Figura 5.5: Diferencia tasas de nacimiento y muerte. Primer ejemplo terapia adaptativa

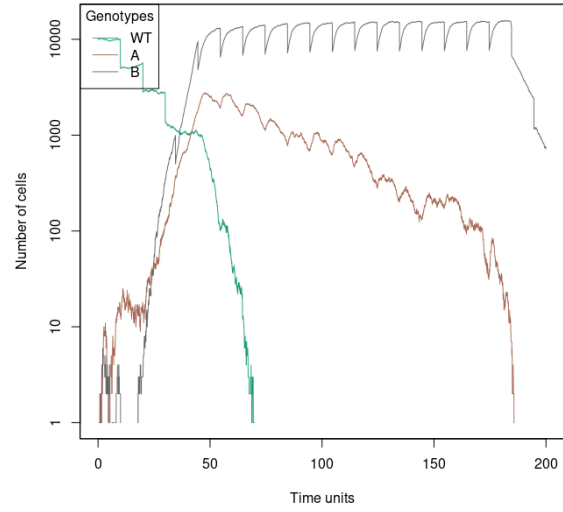


Figura 5.6: Evolución poblacional. Primer ejemplo terapia adaptativa

establecemos en qué consistiría esta terapia adaptativa aplicable en la realidad, de modo similar a los ejemplos presentes en los artículos de Hansen [4][5][20], pero con un escenario bastante más complejo que combine varios tratamientos que se deban ajustar según su agresividad y eficacia contra el tumor:

1. Cada cierto tiempo el paciente tiene una revisión dónde se analiza el progreso de su tumor.
2. Si se detectan unos valores anormales en las poblaciones de genotipos mutados se comienza un tratamiento base para tratar de frenar el avance de este.
3. Si se observa un crecimiento anormalmente grande de un tipo determinado de células se cambia el tratamiento que se estaba aplicando hasta el momento por otro más enfocado a las células de ese tipo.
4. Si el tumor sobrepasa un cierto tamaño preestablecido se plantea la necesidad de realizar una intervención quirúrgica para tratar de eliminarlo.

Este escenario se corresponde a un tratamiento que aplica la técnica “**SMART**” (Sequential, Multiple Assignment, Randomized Trial), dónde cada cierto tiempo se evalúa el tratamiento a dar a un paciente en función de cómo está respondiendo a estos.

Para este ejemplo además consideraremos preocupantes las células de genotipos distintos al **WT**, pues este es el inicial sin mutaciones, y, por tanto, no se considera peligroso.

Todo el código para reproducir este ejemplo se encuentra en el apéndice A.4.

Procedemos entonces a tratar de simular este escenario con tres genotipos **WT**, **A** y **B**, para ello definimos en primer lugar una serie de variables de usuario que serán:

measure	Flag que determina si toca realizar una medición o no (revisión del paciente)
lastTime	Almacena el último tiempo en que se ha realizado una medición
lastMeasuredA	Valor de la población del genotipo A en la última medición
lastMeasuredB	Valor de la población del genotipo B en la última medición
previousA	Valor de la población del genotipo A en la medición anterior (permite luego calcular las diferencias para ver el crecimiento)
previousB	Valor de la población del genotipo B en la medición anterior (permite luego calcular las diferencias para ver el crecimiento)
treatment	Variable que determinará el tratamiento a aplicar

Tabla 5.1: Variables del segundo ejemplo de terapia adaptativa.

Estas las modificaremos de acuerdo a las siguientes reglas:

1.	Si "current time (T)" - lastTime <10 (no han pasado 10 segundos): measure = 0 (no hay que medir)
2.	Si T - lastTime >10 (han pasado 10 segundos): measure = 1 (hay que medir) y lastTime = T
3.	Si measure == 1 (hay que medir): previousA = lastMeasuredA ; previousB = lastMeasuredB ; lastMeasuredA = n_A y lastMeasuredB = n_B (almacenamos los datos medidos).
4.	Si no se cumple ninguna de las condiciones siguientes (Si true): treatment = 0.
5.	Si lastMeasuredA + lastMeasuredB >100 (tumor detectable): treatment = 1
6.	Si lastMeasuredA - previousA >500 (A ha crecido mucho): treatment = 2
7.	Si lastMeasuredB - previousB >500 (B ha crecido mucho): treatment = 3
8.	Si lastMeasuredA - previousA >500 AND lastMeasuredB - previousB >500 (A y B han crecido mucho): treatment = 4

Tabla 5.2: Reglas del segundo ejemplo de terapia adaptativa.

Ahora podemos definir las intervenciones en base al valor de **treatment** del siguiente modo:

```
interventions <- list(
  list(ID = "basicTreatment",
        Trigger = "treatment == 1",
        WhatHappens = "N = 0.8*N",
        Periodicity = 10,
        Repetitions = Inf
  ), list(ID = "treatmentOverA",
        Trigger = "treatment == 2 or treatment == 4",
        WhatHappens = "n_B = n_B*0.3",
        Periodicity = 20,
        Repetitions = Inf
  ), list(ID = "treatmentOverB",
        Trigger = "treatment == 3 or treatment == 4",
        WhatHappens = "n_B = n_B*0.3",
        Periodicity = 20,
        Repetitions = Inf
  ), list(ID = "intervention",
        Trigger = "lastMeasuredA+lastMeasuredB > 5000",
```

```

WhatHappens = "N = 0.1*N" ,
Periodicity = 70 ,
Repetitions = Inf
)
)

```

Establecemos que el tratamiento básico se puede realizar cada 10 unidades de tiempo, los específicos sobre **A** y **B** al ser más agresivos establecemos que sólo se pueden realizar cada 20 unidades de tiempo, por último, la intervención quirúrgica no se puede realizar de forma muy habitual, por lo que establecemos que puede ocurrir cada 70 unidades, y se realizará cuando el conjunto de células peligrosas supere 5000. Esto es muy interesante ya que nos permite ajustarnos a la realidad, dónde ciertos tratamientos requieren que el paciente realice tiempos de descanso, pues son excesivamente agresivos y provocan numerosos efectos adversos.

Con todo esto comenzamos la simulación, obteniendo el resultado poblacional de la figura 5.7.

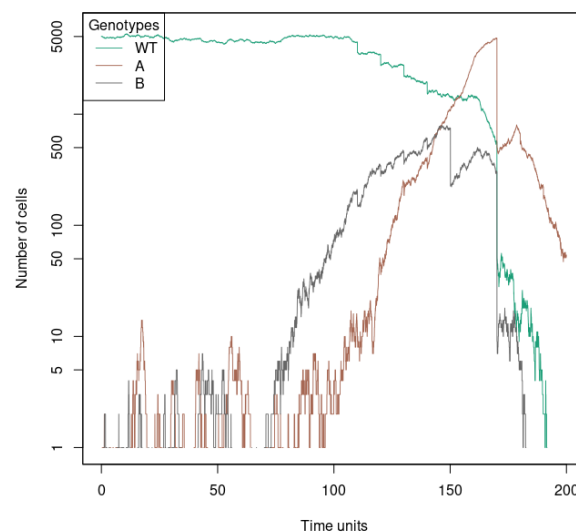


Figura 5.7: Evolución poblacional. Segundo ejemplo terapia adaptativa

Podemos ver como a partir de aproximadamente el tiempo 100 comienza a haber pequeñas bajadas bruscas en todos los genotipos al mismo tiempo, correspondientes al tratamiento base. Luego alrededor del tiempo 150 el genotipo **B** aumenta demasiado y sufre una bajada más grande por su tratamiento específico; y, finalmente, alrededor del tiempo 160-170 las poblaciones **A** y **B** han crecido lo suficiente como para que se lleve a cabo una intervención, por lo que se desploman todas las poblaciones.

Este es por tanto un clarísimo ejemplo de una terapia adaptada al progreso del tumor, y por tanto vemos que se ha logrado con éxito la simulación de la terapia adaptativa.

6

Conclusiones y trabajo futuro

6.1. Conclusiones

Durante el desarrollo de este trabajo se ha implementado en el programa **OncoSimulR**[6] un módulo para permitir la creación y tratamiento de variables arbitrariamente complejas definidas por el usuario para trazar durante la simulación. Además se ha ampliado la funcionalidad que ya existía para simulación de intervenciones y tratamientos, permitiendo que estas empleen las variables de usuario creadas, logrando así reproducir conjuntos complejos de tratamientos aplicables en función de la evolución del conjunto de células tumorales; esto permite, entre otros, y de forma muy destacable, emular la terapia adaptativa [4][5][20].

Desde un punto de vista académico, este trabajo ha permitido al autor familiarizarse y sentirse cómodo con los lenguajes R y C++ que no dominaba, así como ampliar su conocimiento acerca de simulación genética, y aprender sobre los algoritmos específicos de simulación tumoral. Además, ha permitido que este adquiriera nociones básicas de genética y medicina relacionadas directamente con los procesos tumorales, abriéndolo a un posible campo de interés en el futuro.

6.2. Líneas de trabajo futuro

La funcionalidad desarrollada en este proyecto abre camino a una serie de posibles mejoras y funcionalidad futura. Algunas posibles mejoras que se podrían plantear son:

- Ampliar el número de variables que se pueden emplear para definir las variables de

usuario, haciéndolas aún más libres y permitiendo prácticamente cualquier función que se le pueda ocurrir al usuario.

- Permitir la modificación de las tasas de nacimiento, muerte y mutación a través de las intervenciones. Este es un cambio que, con las mejoras ya realizadas no es complejo, y dotaría al programa de la capacidad de simular escenarios aún más arbitrarios convirtiéndolo en un simulador general de evolución asexual, aplicable a otros entornos más allá del cáncer tales como evolución en poblaciones de bacterias, en las que estas modificaciones son más comunes. También ampliaría los tratamientos simulables en el cáncer añadiendo la posibilidad de simular algunos antitumorales que aumentan las tasas de muerte o disminuyen la de nacimiento.
- Implementar funciones para representar gráficamente de forma eficiente los valores en el tiempo de las variables de usuario, puesto que por el momento la salida es simplemente en forma de tabla de datos y realizar una gráfica con estos es ligeramente engorroso.

Bibliografía

- [1] Instituto Nacional del Cáncer de España (NIH). *¿Qué es el cáncer?* URL: <https://www.cancer.gov/espanol/cancer/naturaleza/que-es>.
- [2] *Defunciones según la causa de muerte. Año 2020.* Instituto Nacional de Estadística de España. 2021. URL: https://www.ine.es/prensa/edcm_2020.pdf.
- [3] R. A. Gatenby, A. S. Silva, R. J. Gillies y B. R. Frieden. *Adaptive Therapy*. National Library of Medicine of the USA. Cancer Res. 2009 Jun 1; 69(11):4894-4903., 2009. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3728826/>.
- [4] E. Hansen y A. F. Read. *Modifying Adaptive Therapy to Enhance Competitive Suppression*. Multidisciplinary Digital Publishing Institute, 2020. URL: <https://www.mdpi.com/2072-6694/12/12/3556>.
- [5] E. Hansen y A. F. Read. *Cancer therapy: Attempt cure or manage drug resistance?* Wiley Online Library, 2020. URL: <https://onlinelibrary.wiley.com/doi/10.1111/eva.12994>.
- [6] R. Díaz Uriarte. *OncoSimulR: genetic simulation with arbitrary epistasis and mutator genes in asexual populations*. International Society for Computational Biology. Oxford Academic Bioinformatics, 2017. URL: <https://academic.oup.com/bioinformatics/article/33/12/1898/2982052>.
- [7] *Bioconductor: open source software for bioinformatics*. URL: <https://www.bioconductor.org/>.
- [8] I. Bozic, T. Antal, H. Ohtsuki, H. Carter, D. Kim, S. Chen, R. Karchin, K. W. Kinzler, B. Vogelstein y M. A. Nowak. *Accumulation of driver and passenger mutations during tumor progression*. Proceedings of the National Academy of Sciences of the United States of America 107 (September):18545–50, 2010. URL: <https://www.pnas.org/doi/full/10.1073/pnas.1010978107>.
- [9] C. D. McFarland, K. S. Korolev, G. V. Kryukovand S. R. Sunyaev y L. A. Mirny. *Impact of deleterious passenger mutations on cancer progression*. Proceedings of the National Academy of Sciences vol. 110 no. 8 pp. 2910–2915, 2013. URL: <https://www.pnas.org/doi/full/10.1073/pnas.1213968110>.
- [10] A. González Klein. *Simulación de modelos evolutivos en cáncer: Nuevos modelos de crecimiento*. Trabajo de fin de grado. Universidad Autónoma de Madrid., 2021.
- [11] W. H. Mather, J. Hasty y L. S. Tsimring. *Fast stochastic algorithm for simulating evolutionary population dynamics*. Bioinformatics (Oxford, England) 28 (9):1230–8, 2012. URL: <https://doi.org/10.1093/bioinformatics/bts130>.

- [12] D. T. Gillespie. *Exact stochastic simulation of coupled chemical reactions*. The Journal of Physical Chemistry, vol. 81, no. 25, 1977. URL: <https://www.caam.rice.edu/~cox/gillespie.pdf>.
- [13] M. A. Gibson y J. Bruck. *Efficient Exact Stochastic Simulation of Chemical Systems with Many Species and Many Channels*. American Chemical Society, 2000. URL: <https://pubs.acs.org/doi/abs/10.1021/jp993732q>.
- [14] J. Muñoz Haro. *Simulación de modelos evolutivos en cáncer: Intervenciones flexibles y condicionales en el proceso evolutivo*. Trabajo de fin de grado. Universidad Autónoma de Madrid., 2021.
- [15] *The R Project for Statistical Computing*. URL: <https://www.r-project.org/>.
- [16] J. López Cano. *Código OncoSimulR, terapia adaptativa y variables de usuario*. URL: <https://github.com/JavierLopezC/OncoSimul/tree/adaptive-ther>.
- [17] *Rcpp: Seamless R and C++ integration*. URL: <https://www.rcpp.org/>.
- [18] A. Partow. *C++ Mathematical Expression Library (ExprTk)*. URL: <https://github.com/ArashPartow/exprtk>.
- [19] *Testthat*. URL: <https://testthat.r-lib.org/>.
- [20] E. Hansen, J. Karslake, R. J. Woods, A. F. Read y K. B. Wood. *Antibiotics can be used to contain drug-resistant bacteria by maintaining sufficiently large sensitive populations*. PLOS Biology, 2020. URL: <https://journals.plos.org/plosbiology/article?id=10.1371/journal.pbio.3000713>.

Glosario

- cáncer** Conjunto de enfermedades caracterizadas por que un conjunto de células comienza a reproducirse de forma muy rápida produciendo un crecimiento anormal, en ocasiones además pueden desplazarse a otros tejidos del cuerpo. 1
- epistasia** Fenómeno por el cual la presencia de un determinado gen mutado afecta a la viabilidad de otro gen, haciendo que el fitness de un gen se vea afectado por la presencia de otro gen mutado en el mismo genotipo. 6
- fitness** Métrica que determina el grado de adaptación de una determinada célula al entorno en que se encuentra, y, por tanto, determina las posibilidades que tiene de sobrevivir, reproducirse o morir modificando así la población de su genotipo. En ocasiones está definida por varios parámetros como las tasas de nacimiento y muerte. 2, 6, 9, 10, 32, 34
- gen** Unidad básica de material genético. 6, 15
- genotipo** Conjunto de genes que compone la totalidad del material genético de una célula. IX, 2, 6–10, 14, 15, 18, 19, 22, 23, 25–27, 30–37, 39
- mutación** Alteración permanente de alguno de los genes que compone el genotipo de una célula. 2, 7, 8, 25–27, 32, 37
- simulador de progresión tumoral** Programa desitinado a la simulación de la evolución de poblaciones de células cancerígenas con reproducción asexual dadas unas condiciones iniciales. Tienen por finalidad el estudio de los tumores y la predicción de los escenarios futuros del tumor. 2
- tasa de muerte** Tasa que se emplea en un distribución exponencial para determinar el tiempo en que se producirá la siguiente muerte de una célula de este genotipo. 6, 8, 9, 14
- tasa de mutación** Tasa que se emplea en un distribución exponencial para determinar el tiempo en que se producirá la próxima mutación de una célula de este genotipo. 6, 9, 15
- tasa de nacimiento** Tasa que se emplea en un distribución exponencial para determinar el tiempo en que se producirá el siguiente nacimiento de una célula de este genotipo. IX, 6–9, 14, 25–27, 29, 33–36

tumor Conjunto de células anormalmente grande que se produce por la multiplicación más rápida de lo normal de un conjunto de células. En ocasiones son producidos por cáncer, en cuyo caso pueden invadir otros tejidos. 1, 2, 5, 6, 10, 13, 15, 16, 37–39

Apéndices



Código para reproducir los ejemplos de la sección Resultados

Todos estos ejemplos así como algunos extra están incluidos en la sección correspondiente a las variables de usuario y terapia adaptativa de la viñeta de OncoSimulR. Se puede encontrar en el link <https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/vignettes/OncoSimulR.Rmd> a partir de la línea 12801.

A.1. Ejemplo de uso de variables de usuario para controlar la proporción de cada genotipo en la población total

<https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/vignettes/OncoSimulR.Rmd> línea 12900.

Código para reproducir el primer ejemplo del punto 5.3.1:

```
dfuv2 <- data.frame(Genotype = c("WT", "B", "A", "B", "A", "C", "A"),  
                    Fitness = c("0*n_",  
                                "1.5",  
                                "1.002",  
                                "1.003",  
                                "1.004"))  
  
afuv2 <- allFitnessEffects(genotFitness = dfuv2,  
                           frequencyDependentFitness = TRUE,
```

```

frequencyType = "abs")

userVars <- list(
  list(Name = "genAProp",
        Value = 0.5),
  list(Name = "genBProp",
        Value = 0.5),
  list(Name = "genABProp",
        Value = 0.0),
  list(Name = "genACProp",
        Value = 0.0)
)

userVars <- createUserVars(userVars)

rules <- list(
  list(ID = "rule_1",
        Condition = "TRUE",
        Action = "genBProp = n_B/N"),
  list(ID = "rule_2",
        Condition = "TRUE",
        Action = "genAProp = n_A/N"),
  list(ID = "rule_3",
        Condition = "TRUE",
        Action = "genABProp = n_A_B/N"),
  list(ID = "rule_4",
        Condition = "TRUE",
        Action = "genACProp = n_A_C/N")
)

rules <- createRules(rules, afuv2)

set.seed(1)
uvex2 <- oncoSimulIndiv(
  afuv2,
  model = "McFLD",
  mu = 1e-4,
  sampleEvery = 0.01,
  initSize = c(20000, 20000),
  initMutant = c("A", "B"),
  finalTime = 10,
  onlyCancer = FALSE,
  userVars = userVars,
  rules = rules
)

plot(uvex2, show = "genotypes", type = "line")

```

```
plot(unlist(uvex2$other$userVarValues) [c(FALSE, FALSE, FALSE, FALSE,
  ↪ TRUE)], unlist(uvex2$other$userVarValues) [c(TRUE, FALSE,
  ↪ FALSE, FALSE, FALSE)], xlab="Time", ylab="Proportion", ylim=c
  ↪ (0,1), type="l", col="purple")
lines(unlist(uvex2$other$userVarValues) [c(FALSE, FALSE, FALSE, FALSE
  ↪ , TRUE)], unlist(uvex2$other$userVarValues) [c(FALSE, TRUE,
  ↪ FALSE, FALSE, FALSE)], type="l", col="#E6AB02")
lines(unlist(uvex2$other$userVarValues) [c(FALSE, FALSE, FALSE, FALSE
  ↪ , TRUE)], unlist(uvex2$other$userVarValues) [c(FALSE, FALSE,
  ↪ TRUE, FALSE, FALSE)], type="l", col="#1B9E77")
lines(unlist(uvex2$other$userVarValues) [c(FALSE, FALSE, FALSE, FALSE
  ↪ , TRUE)], unlist(uvex2$other$userVarValues) [c(FALSE, FALSE,
  ↪ FALSE, TRUE, FALSE)], type="l", col="#666666")
legend(0,1,legend=c("genABProp", "genACProp", "genAProp", "genBProp")
  ↪ , col=c("purple", "#E6AB02", "#1B9E77", "#666666"), lty= 1:2)
```

A.2. Ejemplo de uso de variables de usuario para calcular la diferencia entre tasa de nacimiento y muerte de los genotipos

<https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/vignettes/OncoSimulR.Rmd> línea 12974.

Código para reproducir el segundo ejemplo del punto 5.3.1.:

```
dfuv3 <- data.frame(Genotype = c("WT", "A", "B"),
  Fitness = c("1",
    "1 + 0.2 * (n_B > 10)",
    ".9 + 0.4 * (n_A > 10)"
  ))
afuv3 <- allFitnessEffects(genotFitness = dfuv3,
  frequencyDependentFitness = TRUE)

userVars <- list(
  list(Name = "genWTRateDiff",
    Value = 0.5
  ), list(Name = "genARateDiff",
    Value = 0.5
  ), list(Name = "genBRateDiff",
    Value = 0.0
  )
)

userVars <- createUserVars(userVars)
```

```

rules <- list(
  list(ID = "rule_1",
    Condition = "TRUE",
    Action = "genWTRateDiff = b_-d_"
  ), list(ID = "rule_2",
    Condition = "TRUE",
    Action = "genARateDiff = b_1-d_1"
  ), list(ID = "rule_3",
    Condition = "TRUE",
    Action = "genBRateDiff = b_2-d_2"
  )
)

rules <- createRules(rules, afuv3)

set.seed(1)

uvex3 <- oncoSimulIndiv(afuv3,
  model = "McFLD",
  onlyCancer = FALSE,
  finalTime = 150,
  mu = 1e-4,
  initSize = 5000,
  keepPhylog = FALSE,
  seed = NULL,
  errorHitMaxTries = FALSE,
  errorHitWallTime = FALSE,
  userVars = userVars,
  rules = rules)

plot(uvex3, show = "genotypes", type = "line")

plot(unlist(uvex3$other$userVarValues) [c(FALSE, FALSE, FALSE, TRUE)
  ↪ ], unlist(uvex3$other$userVarValues) [c(FALSE, FALSE, TRUE,
  ↪ FALSE)], xlab="Time", ylab="Rate Diff", xlim=c(0,150), ylim=c
  ↪ (-0.75,0.75), type="l", col="#1B9E77")
lines(unlist(uvex3$other$userVarValues) [c(FALSE, FALSE, FALSE, TRUE)
  ↪ ], unlist(uvex3$other$userVarValues) [c(TRUE, FALSE, FALSE,
  ↪ FALSE)], type="l", col="#A6761D")
lines(unlist(uvex3$other$userVarValues) [c(FALSE, FALSE, FALSE, TRUE)
  ↪ ], unlist(uvex3$other$userVarValues) [c(FALSE, TRUE, FALSE,
  ↪ FALSE)], type="l", col="#666666")
legend(0,0.75, legend=c("genWTRateDiff", "genARateDiff", "genBRateDiff
  ↪ "), col=c("#1B9E77", "#A6761D", "#666666"), lty= 1:2)

```

A.3. Ejemplo de terapia adaptativa para atacar al genotipo más adaptado

<https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/vignettes/OncoSimulR.Rmd> línea 13117.

Código para reproducir el primer ejemplo del punto 5.3.2:

```
dfat2 <- data.frame(Genotype = c("WT", "A", "B"),
                    Fitness = c("1",
                                "1 + 0.2 * (n_B > 10)",
                                ".9 + 0.4 * (n_A > 10)"
                                ))
afat2 <- allFitnessEffects(genotFitness = dfat2,
                           frequencyDependentFitness = TRUE)

userVars <- list(
  list(Name = "genADiff",
        Value = 0
  ), list(Name = "genBDiff",
          Value = 0
  ), list(Name = "genWTDiff",
          Value = 0
  )
)

userVars <- createUserVars(userVars)

rules <- list(
  list(ID = "rule_1",
        Condition = "TRUE",
        Action = "genADiff = b_1-d_1"
  ), list(ID = "rule_2",
          Condition = "TRUE",
          Action = "genBDiff = b_2-d_2"
  ), list(ID = "rule_3",
          Condition = "TRUE",
          Action = "genWTDiff = b_-d_"
  ), list(ID = "rule_4",
          Condition = "n_A < 1000",
          Action = "genADiff = -1"
  ), list(ID = "rule_5",
          Condition = "n_B < 1000",
          Action = "genBDiff = -1"
  ), list(ID = "rule_6",
          Condition = "n_ < 1000",
          Action = "genWTDiff = -1"
  )
)
```

```

    )
  )

rules <- createRules(rules , afat2)

interventions <- list(
  list(ID = "i1",
    Trigger = "genADiff > genBDiff and genADiff > genWTDiff"
    ↪ " ",
    WhatHappens = "n_A = n_A*0.5",
    Periodicity = 10,
    Repetitions = Inf
  ), list(ID = "i2",
    Trigger = "genBDiff > genADiff and genBDiff > genWTDiff"
    ↪ " ",
    WhatHappens = "n_B = n_B*0.5",
    Periodicity = 10,
    Repetitions = Inf
  ), list(ID = "i3",
    Trigger = "genWTDiff > genADiff and genWTDiff >
    ↪ genBDiff",
    WhatHappens = "n_ = n_*0.5",
    Periodicity = 10,
    Repetitions = Inf
  )
)

interventions <- createInterventions(interventions , afat2)

set.seed(1) ## for reproducibility
atex2 <- oncoSimulIndiv(afat2 ,
  model = "McFLD",
  onlyCancer = FALSE,
  finalTime = 200,
  mu = 1e-4,
  initSize = 5000,
  keepPhylog = FALSE,
  seed = NULL,
  errorHitMaxTries = FALSE,
  errorHitWallTime = FALSE,
  userVars = userVars ,
  rules = rules ,
  interventions = interventions)

plot(atex2 , show = "genotypes" , type = "line")

plot(unlist(atex2$other$userVarValues) [c(FALSE, FALSE, FALSE, TRUE)
  ↪ ], unlist(atex2$other$userVarValues) [c(TRUE, FALSE, FALSE,
  ↪ FALSE)], xlab="Time" , ylab="RateDiff" , ylim=c(-0.75,0.75) , type

```

```

    ↪ = "1", col="#A6761D")
lines(unlist(atex2$other$userVarValues) [c(FALSE, FALSE, FALSE, TRUE)
    ↪ ], unlist(atex2$other$userVarValues) [c(FALSE, TRUE, FALSE,
    ↪ FALSE)] , type="1", col="#666666")
lines(unlist(atex2$other$userVarValues) [c(FALSE, FALSE, FALSE, TRUE)
    ↪ ], unlist(atex2$other$userVarValues) [c(FALSE, FALSE, TRUE,
    ↪ FALSE)] , type="1", col="#1B9E77")
legend(0,0.75,legend=c("genADiff", "genBDiff", "genWTDiff"), col=c("#
    ↪ A6761D", "#666666", "#1B9E77"), lty= 1:2)

```

A.4. Ejemplo de terapia adaptativa en escenario verosímil

<https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/vignettes/OncoSimulR.Rmd> línea 13223.

Código para reproducir el segundo ejemplo del punto 5.3.2:

```

dfat3 <- data.frame(Genotype = c("WT", "A", "B"),
                    Fitness = c("1",
                                "0.8 + 0.2 * (n_B > 10) + 0.1 (n_A >
                                ↪ 10)",
                                "0.8 + 0.25 * (n_B > 10)"
                                ))
afat3 <- allFitnessEffects(genotFitness = dfat3,
                          frequencyDependentFitness = TRUE)

userVars <- list(
  list(Name = "lastMeasuredA",
        Value = 0
  ), list(Name = "lastMeasuredB",
        Value = 0
  ), list(Name = "previousA",
        Value = 0
  ), list(Name = "previousB",
        Value = 0
  ), list(Name = "lastTime",
        Value = 0
  ), list(Name = "measure",
        Value = 0
  ), list(Name = "treatment",
        Value = 0
  )
)

```

```

userVars <- createUserVars(userVars)

rules <- list(
  list(ID = "rule_1",
    Condition = "T - lastTime < 10",
    Action = "measure = 0"
  ), list(ID = "rule_2",
    Condition = "T - lastTime >= 10",
    Action = "measure = 1; lastTime = T"
  ), list(ID = "rule_3",
    Condition = "measure == 1",
    Action = "previousA = lastMeasuredA; previousB =
      ↪ lastMeasuredB; lastMeasuredA = n_A; lastMeasuredB = n
      ↪ _B"
  ), list(ID = "rule_4",
    Condition = "TRUE",
    Action = "treatment = 0"
  ), list(ID = "rule_5",
    Condition = "lastMeasuredA + lastMeasuredB > 100",
    Action = "treatment = 1"
  ), list(ID = "rule_6",
    Condition = "lastMeasuredA - PreviousA > 500",
    Action = "treatment = 2"
  ), list(ID = "rule_7",
    Condition = "lastMeasuredB - PreviousB > 500",
    Action = "treatment = 3"
  ), list(ID = "rule_8",
    Condition = "lastMeasuredA - PreviousA > 500 and
      ↪ lastMeasuredB - PreviousB > 500",
    Action = "treatment = 4"
  )
)

rules <- createRules(rules, afat3)

interventions <- list(
  list(ID = "basicTreatment",
    Trigger = "treatment == 1",
    WhatHappens = "N = 0.8*N",
    Periodicity = 10,
    Repetitions = Inf
  ), list(ID = "treatmentOverA",
    Trigger = "treatment == 2 or treatment == 4",
    WhatHappens = "n_B = n_B*0.3",
    Periodicity = 20,
    Repetitions = Inf
  ), list(ID = "treatmentOverB",
    Trigger = "treatment == 3 or treatment == 4",

```

```

        WhatHappens    = "n_B = n_B*0.3",
        Periodicity    = 20,
        Repetitions    = Inf
    ), list(ID          = "intervention",
        Trigger        = "lastMeasuredA+lastMeasuredB > 5000",
        WhatHappens    = "N = 0.1*N",
        Periodicity    = 70,
        Repetitions    = Inf
    )
)

interventions <- createInterventions(interventions, afat3)

set.seed(1) ## for reproducibility
atex2 <- oncoSimulIndiv(afat3,
    model = "McFLD",
    onlyCancer = FALSE,
    finalTime = 200,
    mu = 1e-4,
    initSize = 5000,
    keepPhylog = FALSE,
    seed = NULL,
    errorHitMaxTries = FALSE,
    errorHitWallTime = FALSE,
    userVars = userVars,
    rules = rules,
    interventions = interventions)

plot(atex2, show = "genotypes", type = "line")

```


B

Documentación de las principales funciones de R

También se ha añadido documentación extensa siguiendo el formato estándar de R para las principales funciones accesibles por el usuario en esta nueva funcionalidad. Esto incluye:

- modificación de la documentación de las funciones *OncoSimulIndiv*, *OncoSimulPop* y *OncoSimulSample* para explicar los nuevos parámetros de la simulación.
<https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/man/oncoSimulIndiv.Rd>.
- Elaboración de documentación extensa y con ejemplos de uso de las funciones *createUserVariables* y *createRules*, que son las accesibles directamente por el usuario para la especificación de estos argumentos.
<https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/man/createUserVars.Rd>.



Cabeceras del TAD de variables de usuario

Aquí se proporciona el código de las cabeceras y las estructuras pertenecientes al TAD que implementa las variables de usuario para facilitar la comprensión de la implementación. Estas cabeceras se corresponden con el documento **user_vars.h**. La implementación detallada de estas funciones se puede ver en https://github.com/JavierLopezC/OncoSimul/blob/adaptive-ther/OncoSimulR/src/user_var.cpp.

```
// we declare the needed symbols for the table
typedef exprtk::symbol_table<double> symbol_table_t;
typedef exprtk::expression<double> expression_t;
typedef exprtk::parser<double> parser_t;

// We define what a rule is
typedef struct {
    std::string id; // identifier of the rule
    std::string condition; // condition that must be true for the
        ⇨ rule to execute
    std::string action; // action to be performed when the
        ⇨ condition is satisfied
} Rule;

// Define a structure with all info asociated with user_vars
typedef struct {
    std::map<std::string, double> userVars; // maps a user
        ⇨ variable name to its current value
    std::vector<Rule> rules; // rules that apply to the group of
        ⇨ user variables
    std::map<std::string, double> mapGenoToPop; // variable that
        ⇨ maps a genotype to its population
```

```

}UserVarsInfo;

// function that creates the UserVarsInfo structure
UserVarsInfo createUserVarsInfo(Rcpp::List Rules,
Rcpp::List userVars,
const fitnessEffectsAll& fitnessEffects,
const std::vector<spParamsP>& popParams,
std::vector<Genotype> Genotypes);

// function that creates a rule in memory
Rule createRule(std::string id,
std::string condition,
std::string action);

// Function that add an user_var to the map of user_vars
UserVarsInfo addUserVar(UserVarsInfo uvif, std::string name,
    ↪ double value);

// Function that add a rule to the array of rules
UserVarsInfo addRule(UserVarsInfo uvif, Rule r);

// function that destroys the rule in memory
UserVarsInfo destroyRule(UserVarsInfo uvif, Rule r);

// function that executes the whole list of rules for the user
    ↪ variables
void executeRules (UserVarsInfo& uvif,
double &currentTime,
std::map<std::string, double> birthMap,
std::map<std::string, double> deathMap,
std::map<std::string, double> mutationMap);

// function that compares two rules
int compareRules(Rule r1, Rule r2);

void printUserVarsInfo(UserVarsInfo uvif);
void printRule(Rule r);

```